

Data Analytics with GenAI By ADDA 247
SQL Mini Project — Retail Store

Solution Queries & Output Screenshots

Submitted By: Arijit Dutta

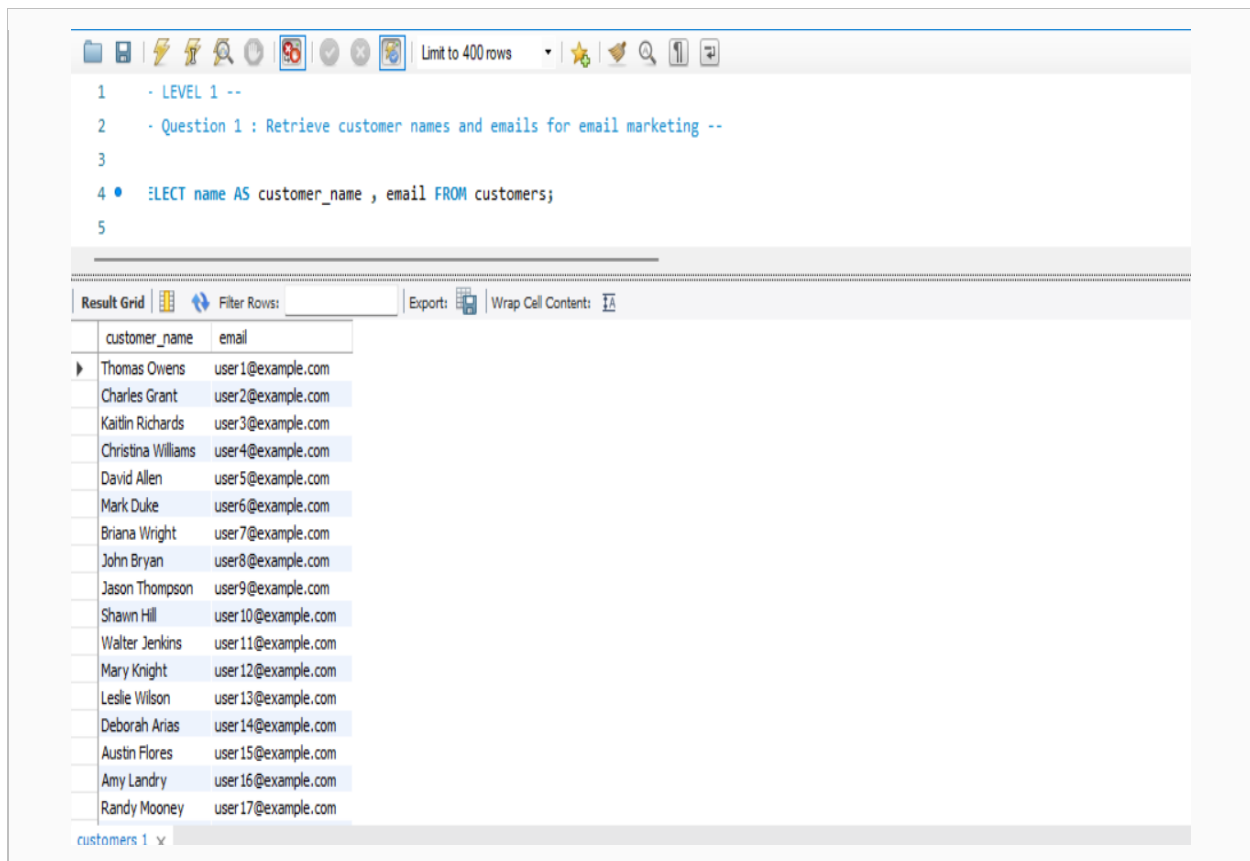
Level 1 — Basic SELECT & Simple Filters

Question 1

Retrieve customer names and emails for email marketing

```
SELECT name AS customer_name, email FROM customers;
```

Output:



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and settings. The query editor displays the following SQL code:

```
1  - LEVEL 1 --  
2  - Question 1 : Retrieve customer names and emails for email marketing --  
3  
4  • SELECT name AS customer_name , email FROM customers;  
5
```

Below the query editor is the 'Result Grid' section. It features a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The results are displayed in a table with two columns: 'customer_name' and 'email'.

customer_name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com
Deborah Arias	user14@example.com
Austin Flores	user15@example.com
Amy Landry	user16@example.com
Randy Mooney	user17@example.com

At the bottom left, there is a label 'customers 1' with a dropdown arrow.

Question 2

View complete product catalog with all available details

```
SELECT * FROM products;
```

Output:

```
4 • SELECT name AS customer_name , email FROM customers;
5
6 -- Question 2 : View complete product catalog with all available details --
7
8 • SELECT * FROM products;
```

	product_id	name	category	price	stock_quantity	added_on
▶	1	Plant No	Home	639.43	152	2024-01-30 06:30:53
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06

Question 3

List all unique product categories

```
SELECT DISTINCT category FROM products;
```

Output:

```
11
12 • SELECT DISTINCT category FROM products;
13
14 -- Question 4 -- Show all products priced above ₹1,000--
```

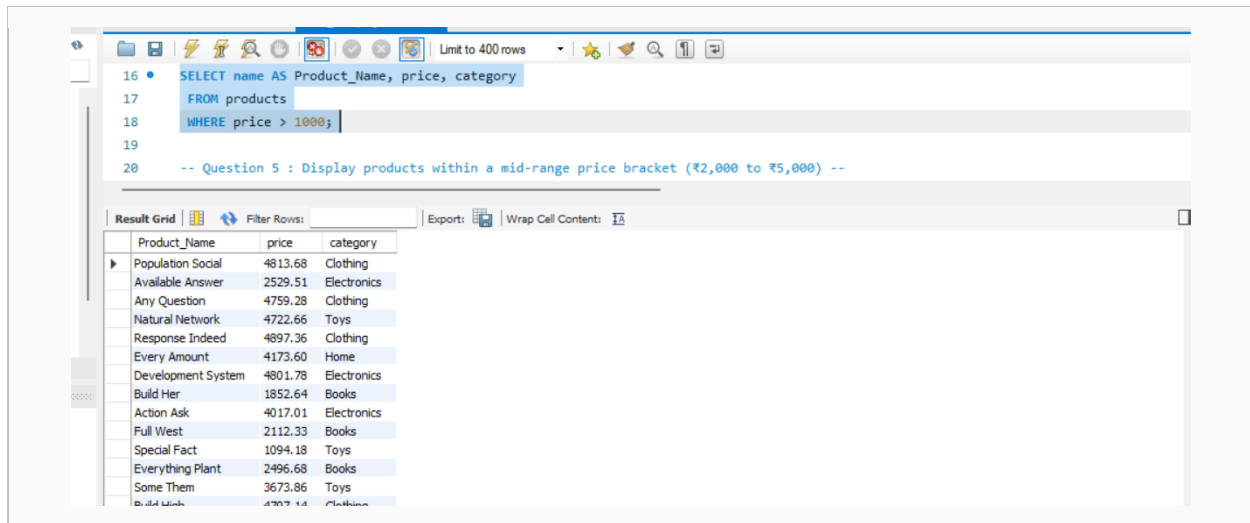
	category
▶	Home
	Clothing
	Electronics
	Toys
	Books

Question 4

Show all products priced above ₹1,000

```
SELECT name AS Product_Name, price, category
FROM products
WHERE price > 1000;
```

Output:



The screenshot shows a database query editor with the following SQL query:

```
16 SELECT name AS Product_Name, price, category
17 FROM products
18 WHERE price > 1000;
19
20 -- Question 5 : Display products within a mid-range price bracket (₹2,000 to ₹5,000) --
```

Below the query, the 'Result Grid' displays the following data:

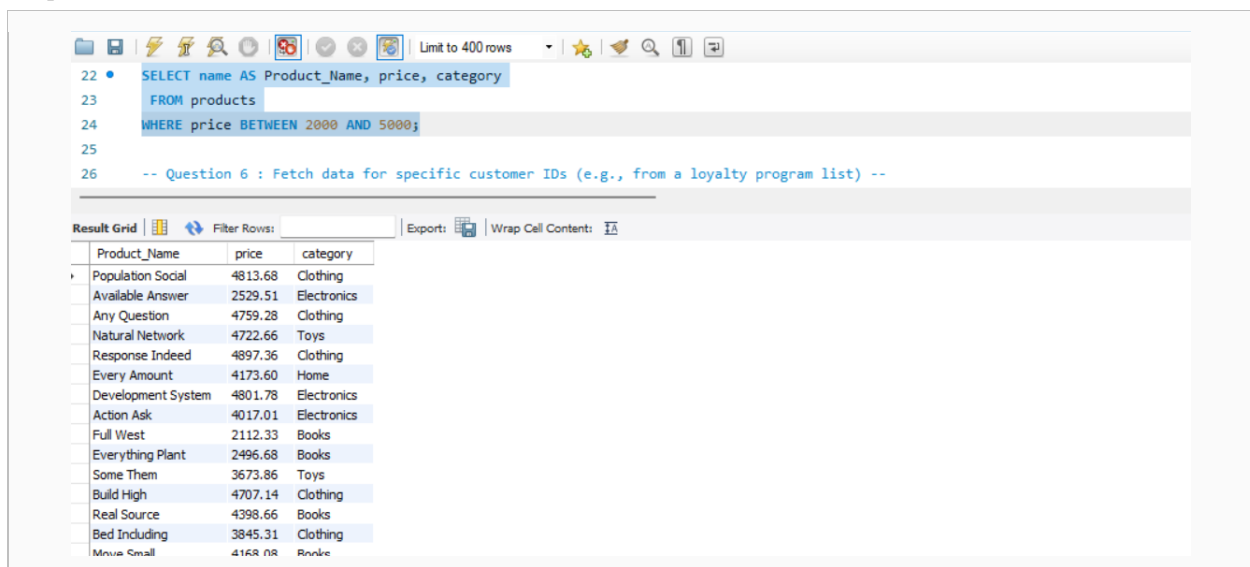
Product_Name	price	category
Population Social	4813.68	Clothing
Available Answer	2529.51	Electronics
Any Question	4759.28	Clothing
Natural Network	4722.66	Toys
Response Indeed	4897.36	Clothing
Every Amount	4173.60	Home
Development System	4801.78	Electronics
Build Her	1852.64	Books
Action Ask	4017.01	Electronics
Full West	2112.33	Books
Special Fact	1094.18	Toys
Everything Plant	2496.68	Books
Some Them	3673.86	Toys
Build High	4707.14	Clothing

Question 5

Display products within a mid-range price bracket (₹2,000 to ₹5,000)

```
SELECT name AS Product_Name, price, category
FROM products
WHERE price BETWEEN 2000 AND 5000;
```

Output:



The screenshot shows a database query editor with the following SQL query:

```
22 SELECT name AS Product_Name, price, category
23 FROM products
24 WHERE price BETWEEN 2000 AND 5000;
25
26 -- Question 6 : Fetch data for specific customer IDs (e.g., from a loyalty program list) --
```

Below the query, the 'Result Grid' displays the following data:

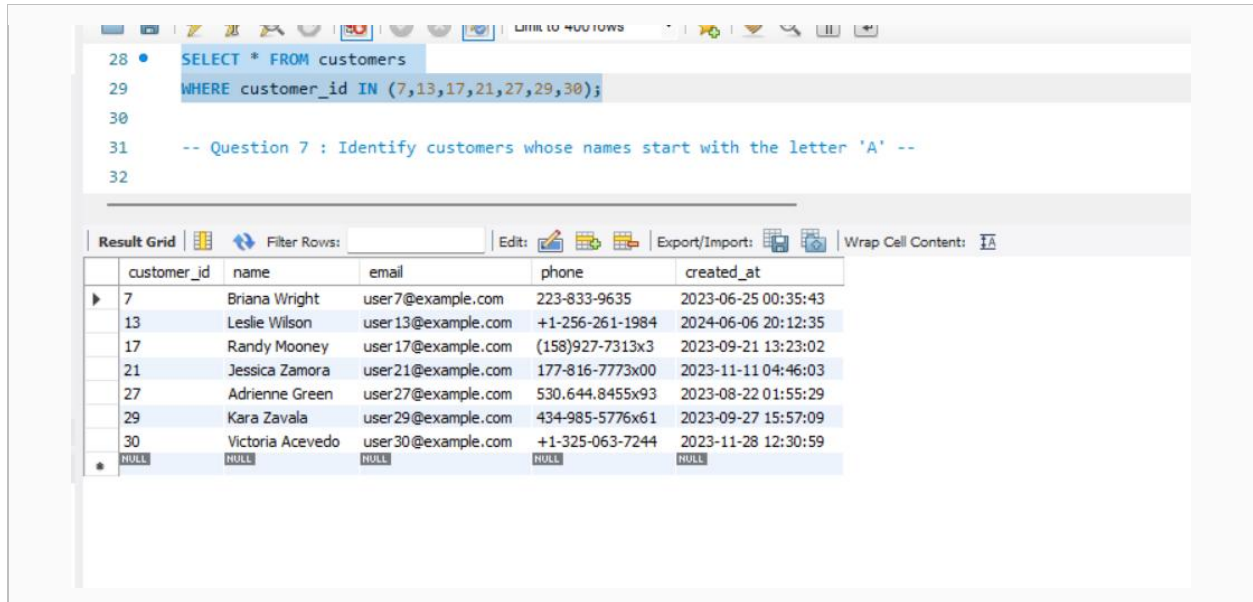
Product_Name	price	category
Population Social	4813.68	Clothing
Available Answer	2529.51	Electronics
Any Question	4759.28	Clothing
Natural Network	4722.66	Toys
Response Indeed	4897.36	Clothing
Every Amount	4173.60	Home
Development System	4801.78	Electronics
Action Ask	4017.01	Electronics
Full West	2112.33	Books
Everything Plant	2496.68	Books
Some Them	3673.86	Toys
Build High	4707.14	Clothing
Real Source	4398.66	Books
Bed Including	3845.31	Clothing
Minus Small	4168.08	Books

Question 6

Fetch data for specific customer IDs (e.g., from a loyalty program list)

```
SELECT * FROM customers
WHERE customer_id IN (7,13,17,21,27,29,30);
```

Output:



The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
28 • SELECT * FROM customers
29 WHERE customer_id IN (7,13,17,21,27,29,30);
30
31 -- Question 7 : Identify customers whose names start with the letter 'A' --
32
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns: customer_id, name, email, phone, and created_at. The results are as follows:

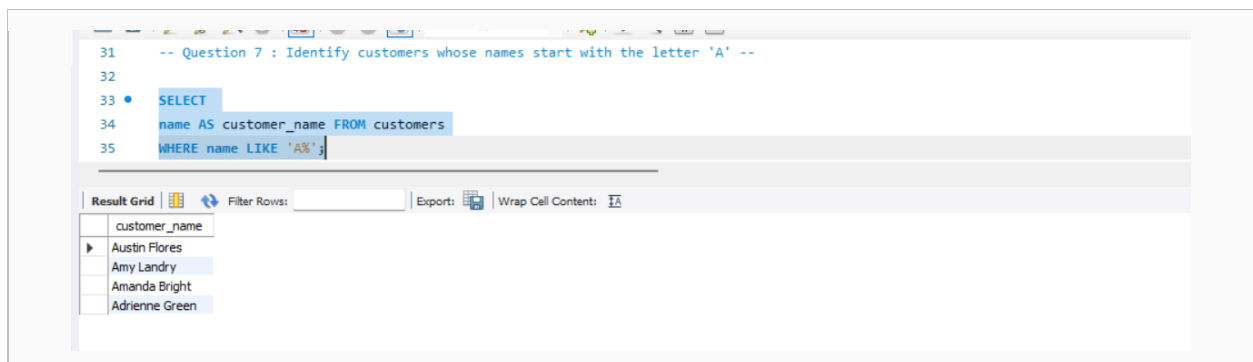
customer_id	name	email	phone	created_at
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35
17	Randy Mooney	user17@example.com	(158)927-7313x3	2023-09-21 13:23:02
21	Jessica Zamora	user21@example.com	177-816-7773x00	2023-11-11 04:46:03
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
29	Kara Zavala	user29@example.com	434-985-5776x61	2023-09-27 15:57:09
30	Victoria Acevedo	user30@example.com	+1-325-063-7244	2023-11-28 12:30:59
NULL	NULL	NULL	NULL	NULL

Question 7

Identify customers whose names start with the letter 'A'

```
SELECT name AS customer_name FROM customers
WHERE name LIKE 'A%';
```

Output:



The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
31 -- Question 7 : Identify customers whose names start with the letter 'A' --
32
33 • SELECT
34 name AS customer_name FROM customers
35 WHERE name LIKE 'A%';
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has a single column: customer_name. The results are as follows:

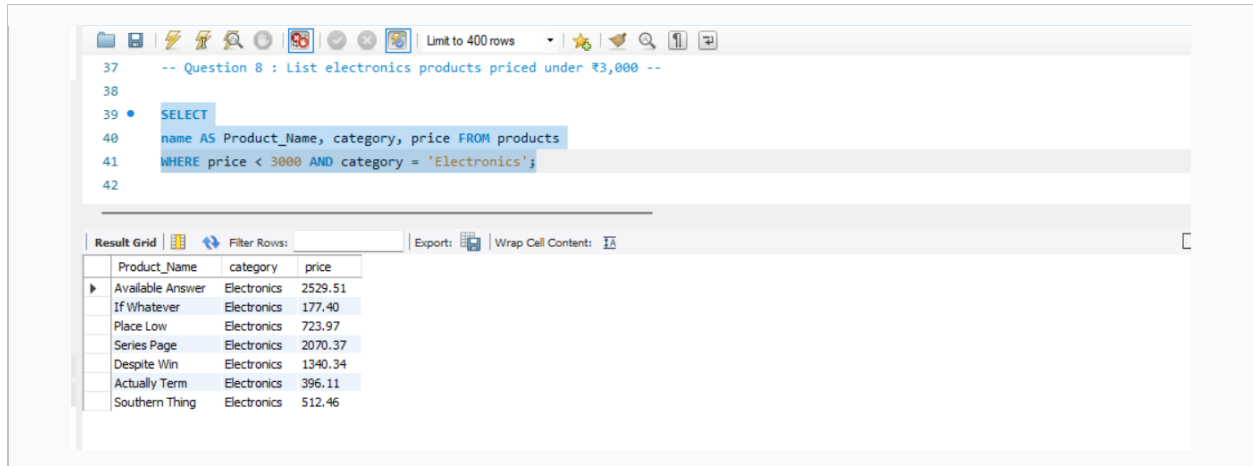
customer_name
Austin Flores
Amy Landry
Amanda Bright
Adrienne Green

Question 8

List electronics products priced under ₹3,000

```
SELECT name AS Product_Name, category, price FROM products
WHERE price < 3000 AND category = 'Electronics';
```

Output:



The screenshot shows a SQL query editor with the following query:

```
-- Question 8 : List electronics products priced under ₹3,000 --
SELECT
name AS Product_Name, category, price FROM products
WHERE price < 3000 AND category = 'Electronics';
```

Below the query, the results are displayed in a grid:

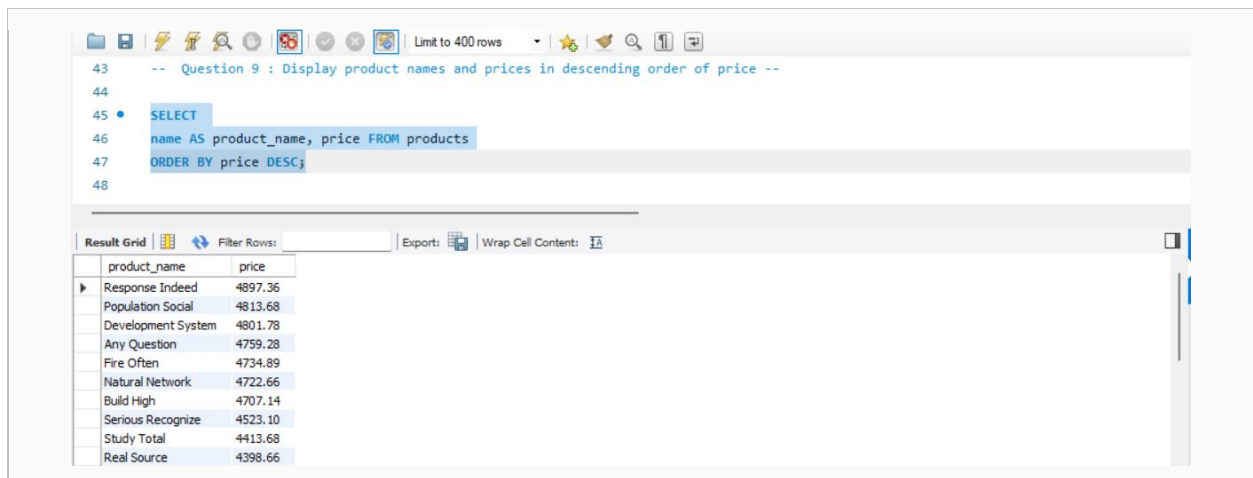
Product_Name	category	price
Available Answer	Electronics	2529.51
If Whatever	Electronics	177.40
Place Low	Electronics	723.97
Series Page	Electronics	2070.37
Despite Win	Electronics	1340.34
Actually Term	Electronics	396.11
Southern Thing	Electronics	512.46

Question 9

Display product names and prices in descending order of price

```
SELECT name AS product_name, price FROM products
ORDER BY price DESC;
```

Output:



The screenshot shows a SQL query editor with the following query:

```
-- Question 9 : Display product names and prices in descending order of price --
SELECT
name AS product_name, price FROM products
ORDER BY price DESC;
```

Below the query, the results are displayed in a grid:

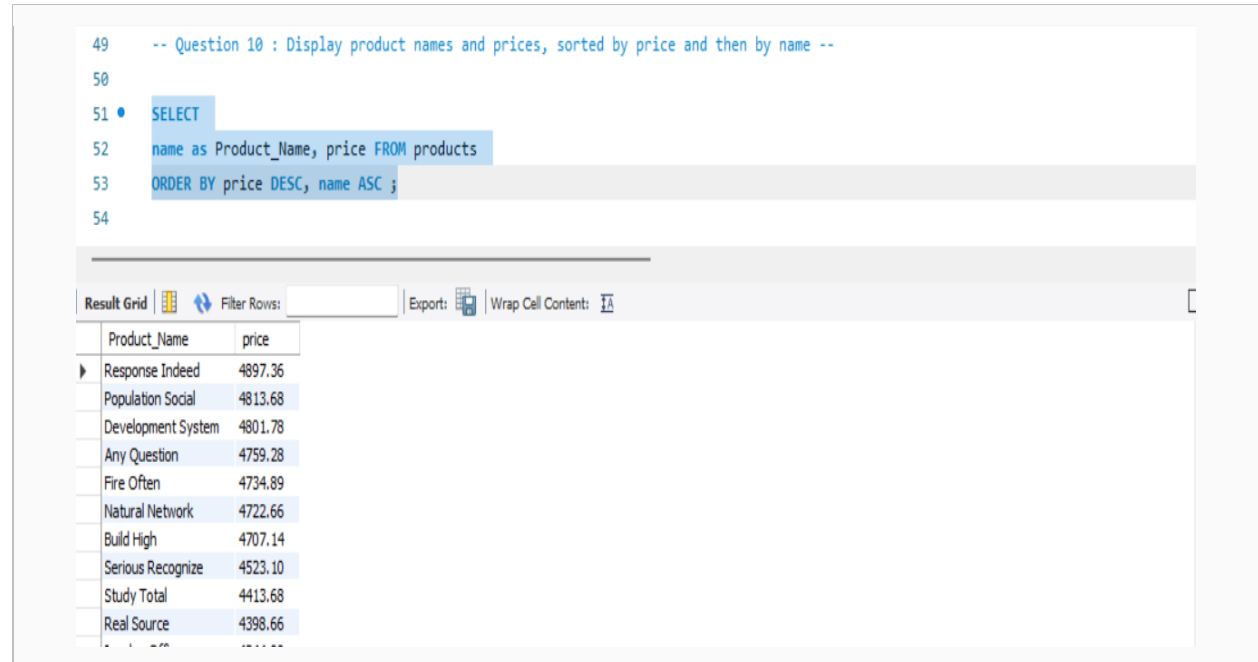
product_name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66

Question 10

Display product names and prices, sorted by price and then by name

```
SELECT name AS Product_Name, price FROM products
ORDER BY price DESC, name ASC;
```

Output:



```
49  -- Question 10 : Display product names and prices, sorted by price and then by name --
50
51  • SELECT
52  name as Product_Name, price FROM products
53  ORDER BY price DESC, name ASC ;
54
```

Product_Name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66

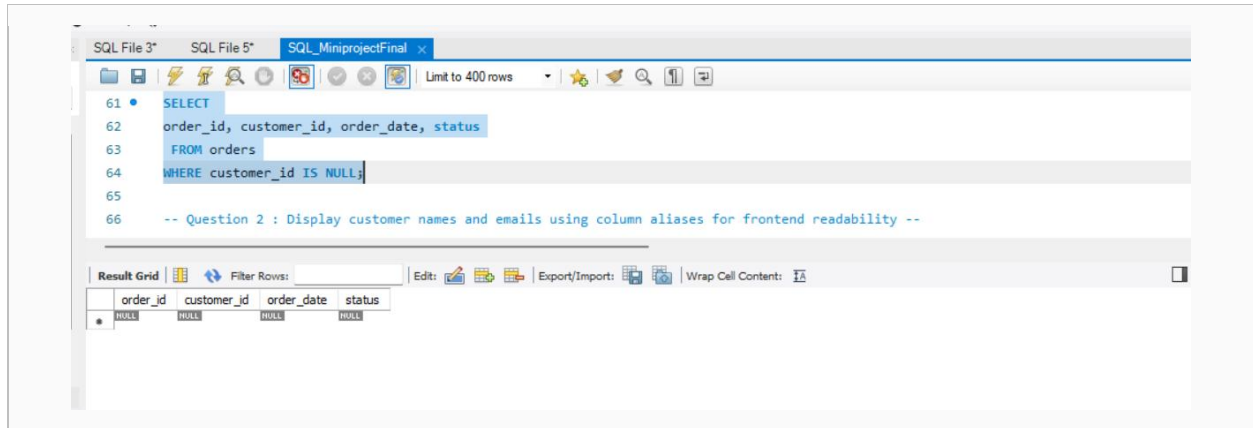
Level 2 — Formatting & Row-level Calculations

Question 1

Retrieve orders where customer information is missing (possibly due to data migration or deletion)

```
SELECT order_id, customer_id, order_date, status
FROM orders
WHERE customer_id IS NULL;
```

Output:



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
61 SELECT
62 order_id, customer_id, order_date, status
63 FROM orders
64 WHERE customer_id IS NULL;
65
66 -- Question 2 : Display customer names and emails using column aliases for frontend readability --
```

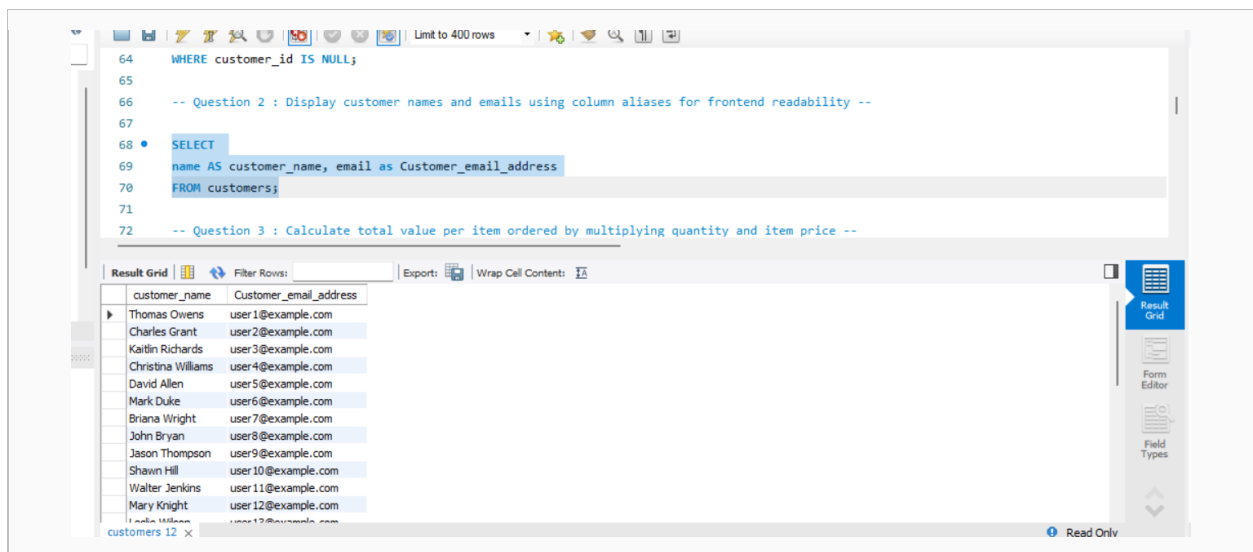
The result grid below the query editor shows the following columns: order_id, customer_id, order_date, and status. The first row of data shows NULL values for all four columns.

Question 2

Display customer names and emails using column aliases for frontend readability

```
SELECT name AS customer_name, email AS Customer_email_address
FROM customers;
```

Output:



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
64 WHERE customer_id IS NULL;
65
66 -- Question 2 : Display customer names and emails using column aliases for frontend readability --
67
68 SELECT
69 name AS customer_name, email as Customer_email_address
70 FROM customers;
71
72 -- Question 3 : Calculate total value per item ordered by multiplying quantity and item price --
```

The result grid below the query editor shows the following columns: customer_name and Customer_email_address. The first 12 rows of data are as follows:

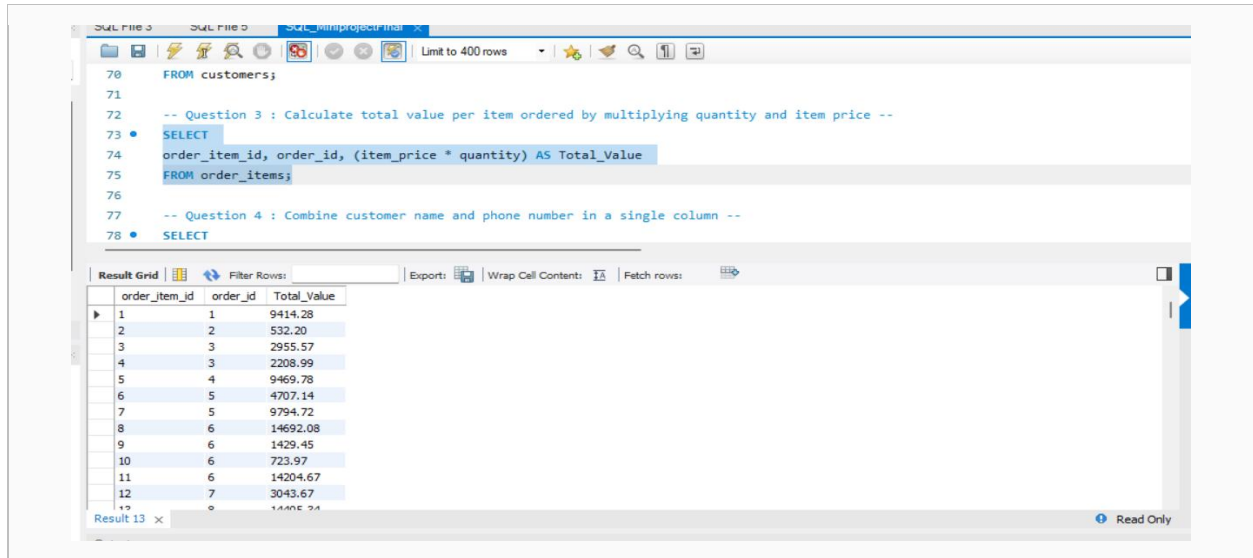
customer_name	Customer_email_address
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com

Question 3

Calculate total value per item ordered by multiplying quantity and item price

```
SELECT order_item_id, order_id, (item_price * quantity) AS Total_Value
FROM order_items;
```

Output:



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
70 FROM customers;
71
72 -- Question 3 : Calculate total value per item ordered by multiplying quantity and item price --
73 • SELECT
74   order_item_id, order_id, (item_price * quantity) AS Total_Value
75 FROM order_items;
76
77 -- Question 4 : Combine customer name and phone number in a single column --
78 • SELECT
```

The result grid displays the following data:

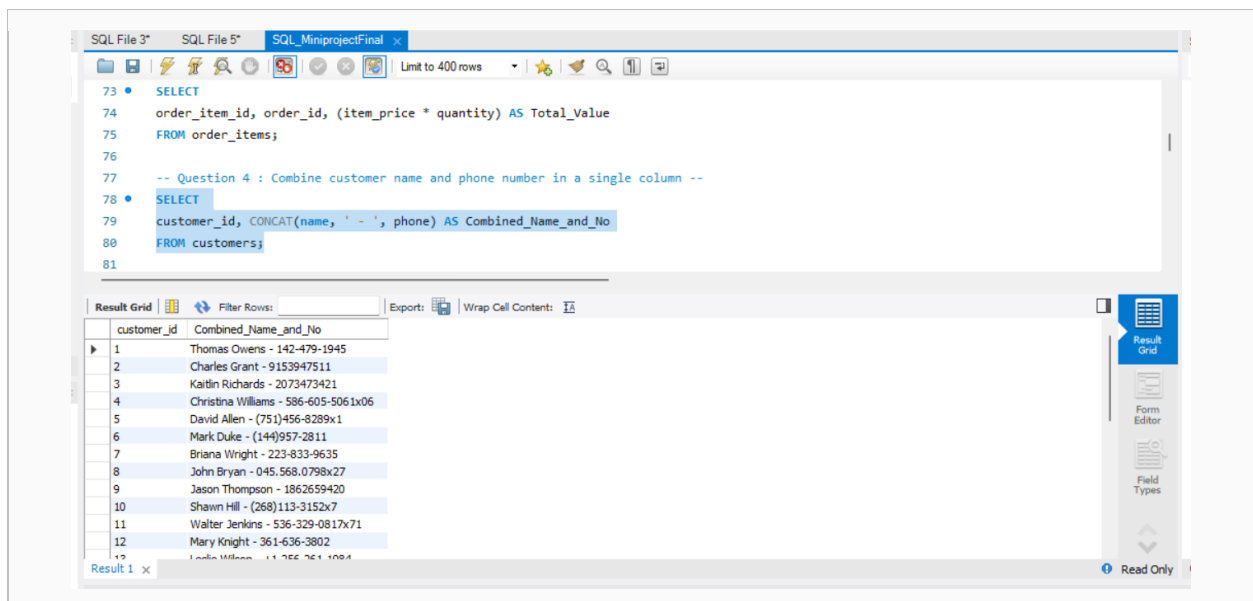
order_item_id	order_id	Total_Value
1	1	9414.28
2	2	532.20
3	3	2955.57
4	3	2208.99
5	4	9469.78
6	5	4707.14
7	5	9794.72
8	6	14692.08
9	6	1429.45
10	6	723.97
11	6	14204.67
12	7	3043.67

Question 4

Combine customer name and phone number in a single column

```
SELECT customer_id, CONCAT(name, ' - ', phone) AS Combined_Name_and_No
FROM customers;
```

Output:



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
73 • SELECT
74   order_item_id, order_id, (item_price * quantity) AS Total_Value
75 FROM order_items;
76
77 -- Question 4 : Combine customer name and phone number in a single column --
78 • SELECT
79   customer_id, CONCAT(name, ' - ', phone) AS Combined_Name_and_No
80 FROM customers;
81
```

The result grid displays the following data:

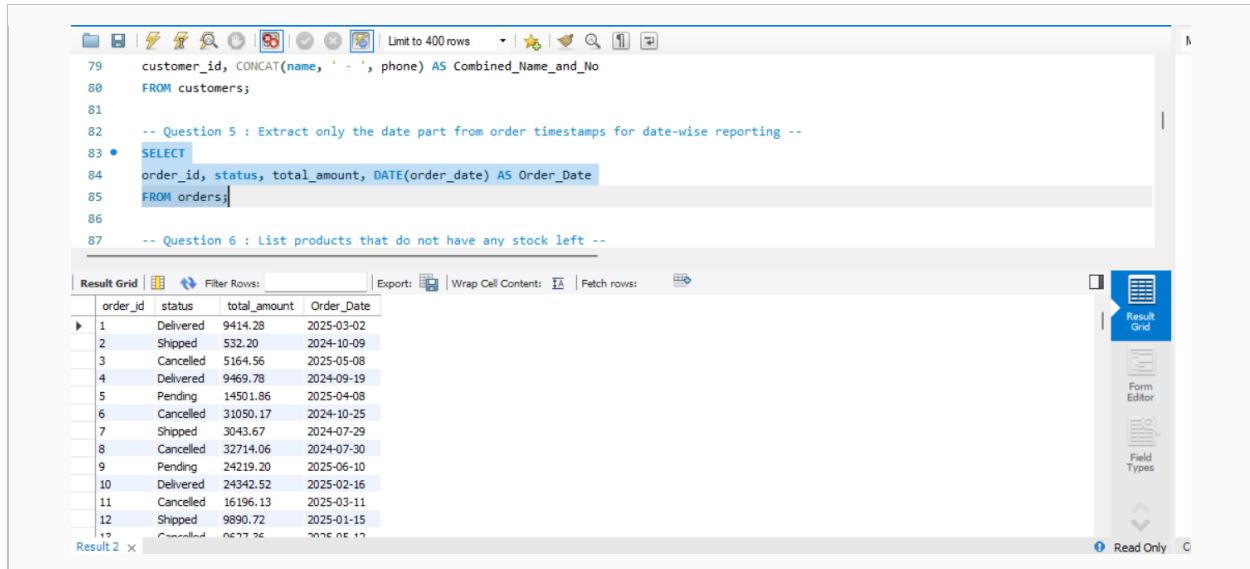
customer_id	Combined_Name_and_No
1	Thomas Owens - 142-479-1945
2	Charles Grant - 9153947511
3	Kaitlin Richards - 2073473421
4	Christina Williams - 586-605-5061x06
5	David Allen - (751)456-8289x1
6	Mark Duke - (144)957-2811
7	Briana Wright - 223-833-9635
8	John Bryan - 045.568.0798x27
9	Jason Thompson - 1862659420
10	Shawin Hill - (268)113-3152x7
11	Walter Jenkins - 536-329-0817x71
12	Mary Knight - 361-636-3802

Question 5

Extract only the date part from order timestamps for date-wise reporting

```
SELECT order_id, status, total_amount, DATE(order_date) AS Order_Date
FROM orders;
```

Output:



```
79 customer_id, CONCAT(name, ' ', phone) AS Combined_Name_and_No
80 FROM customers;
81
82 -- Question 5 : Extract only the date part from order timestamps for date-wise reporting --
83 • SELECT
84 order_id, status, total_amount, DATE(order_date) AS Order_Date
85 FROM orders;
86
87 -- Question 6 : List products that do not have any stock left --
```

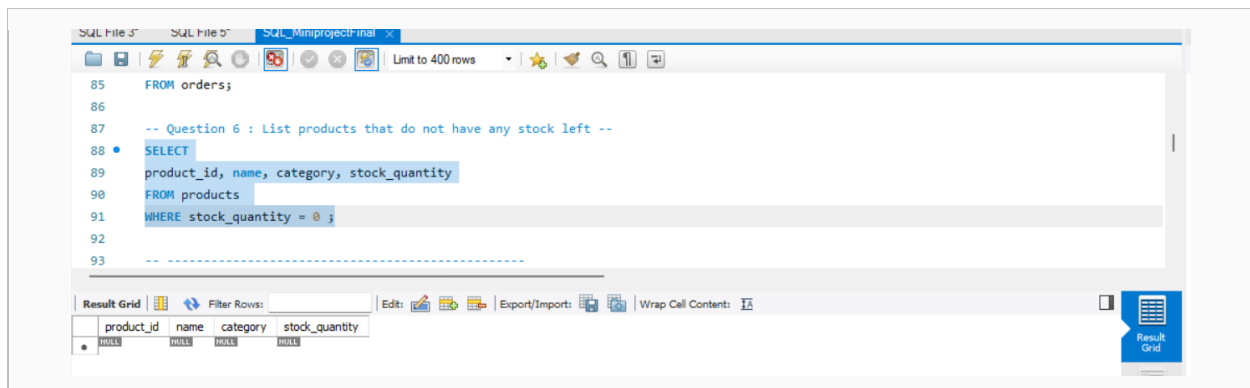
order_id	status	total_amount	Order_Date
1	Delivered	9414.28	2025-03-02
2	Shipped	532.20	2024-10-09
3	Cancelled	5164.56	2025-05-08
4	Delivered	9469.78	2024-09-19
5	Pending	14501.86	2025-04-08
6	Cancelled	31050.17	2024-10-25
7	Shipped	3043.67	2024-07-29
8	Cancelled	32714.06	2024-07-30
9	Pending	24219.20	2025-06-10
10	Delivered	24342.52	2025-02-16
11	Cancelled	16196.13	2025-03-11
12	Shipped	9890.72	2025-01-15

Question 6

List products that do not have any stock left

```
SELECT product_id, name, category, stock_quantity
FROM products
WHERE stock_quantity = 0;
```

Output:



```
85 FROM orders;
86
87 -- Question 6 : List products that do not have any stock left --
88 • SELECT
89 product_id, name, category, stock_quantity
90 FROM products
91 WHERE stock_quantity = 0 ;
92
93 -----
```

product_id	name	category	stock_quantity
1	...	...	0
2	...	...	0
3	...	...	0
4	...	...	0

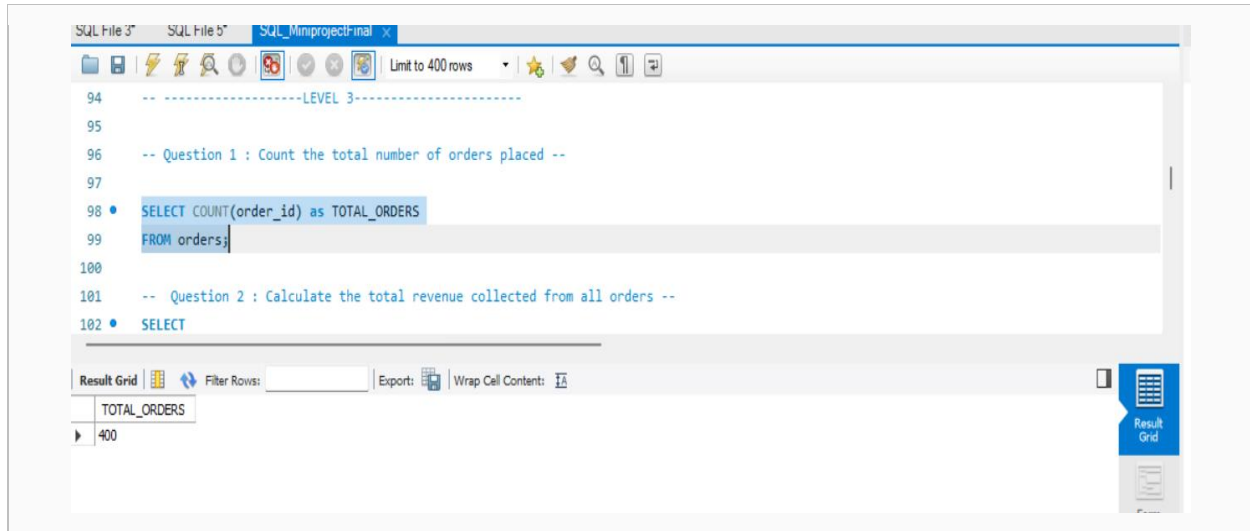
Level 3 — Aggregations & GROUP BY

Question 1

Count the total number of orders placed

```
SELECT COUNT(order_id) AS TOTAL_ORDERS  
FROM orders;
```

Output:



The screenshot shows the SQL Studio interface. The query editor contains the following SQL code:

```
-- -----LEVEL 3-----  
  
-- Question 1 : Count the total number of orders placed --  
  
SELECT COUNT(order_id) as TOTAL_ORDERS  
FROM orders;  
  
-- Question 2 : Calculate the total revenue collected from all orders --  
SELECT
```

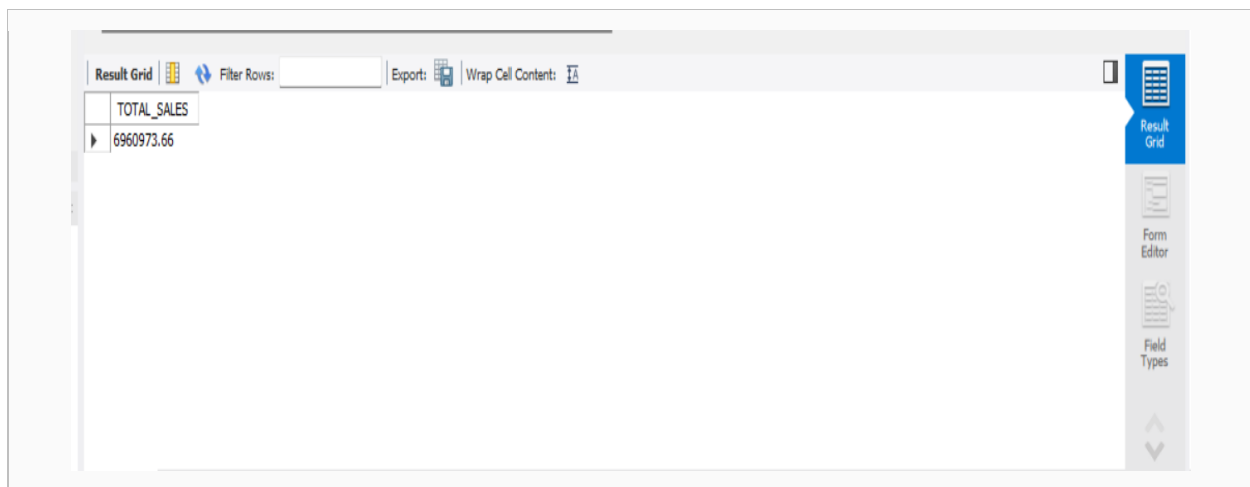
The query is highlighted in blue. Below the editor, the 'Result Grid' tab is active, showing a single row with the column 'TOTAL_ORDERS' and the value '400'.

Question 2

Calculate the total revenue collected from all orders

```
SELECT SUM(total_amount) AS TOTAL_SALES  
FROM orders;
```

Output:



The screenshot shows the SQL Studio interface. The query editor contains the following SQL code:

```
SELECT SUM(total_amount) AS TOTAL_SALES  
FROM orders;
```

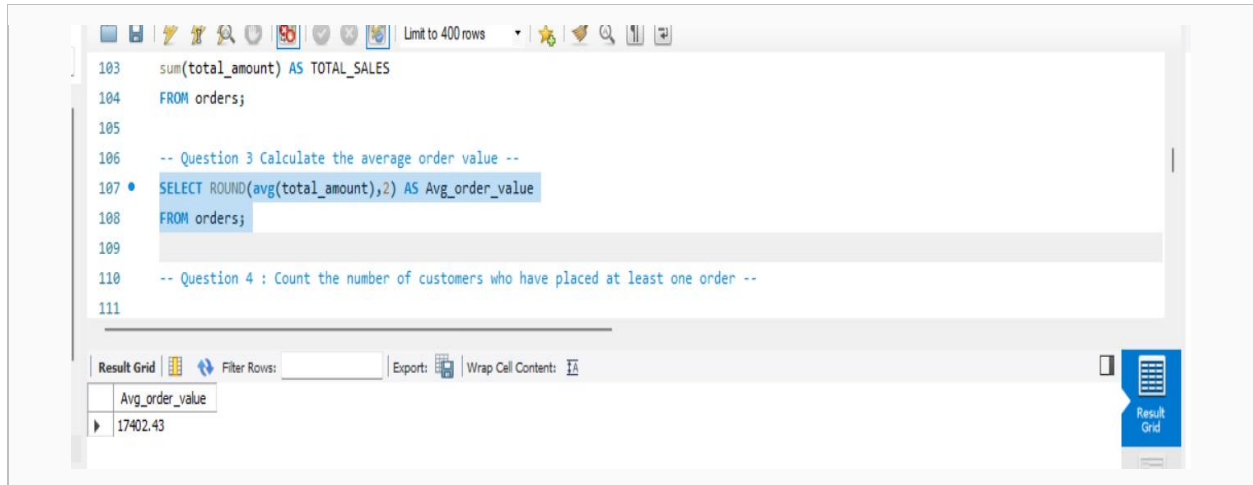
The query is highlighted in blue. Below the editor, the 'Result Grid' tab is active, showing a single row with the column 'TOTAL_SALES' and the value '6960973.66'.

Question 3

Calculate the average order value

```
SELECT ROUND(AVG(total_amount),2) AS Avg_order_value
FROM orders;
```

Output:



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
103 sum(total_amount) AS TOTAL_SALES
104 FROM orders;
105
106 -- Question 3 Calculate the average order value --
107 • SELECT ROUND(avg(total_amount),2) AS Avg_order_value
108 FROM orders;
109
110 -- Question 4 : Count the number of customers who have placed at least one order --
111
```

The result grid at the bottom shows the output of the query:

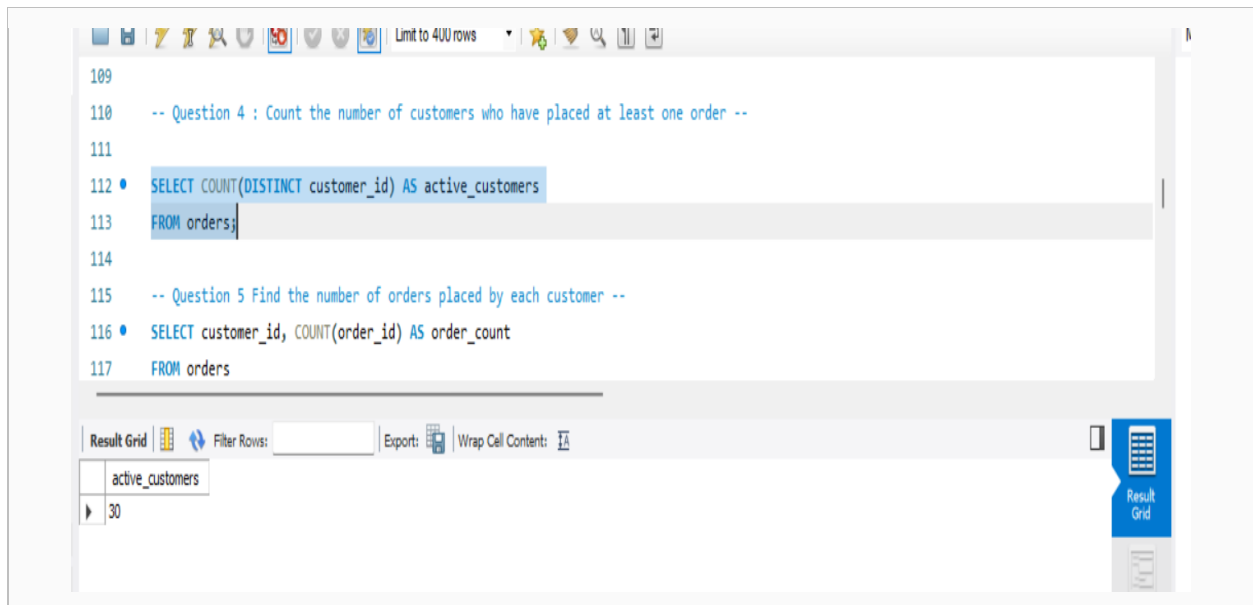
Avg_order_value
17402.43

Question 4

Count the number of customers who have placed at least one order

```
SELECT COUNT(DISTINCT customer_id) AS active_customers
FROM orders;
```

Output:



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
109
110 -- Question 4 : Count the number of customers who have placed at least one order --
111
112 • SELECT COUNT(DISTINCT customer_id) AS active_customers
113 FROM orders;
114
115 -- Question 5 Find the number of orders placed by each customer --
116 • SELECT customer_id, COUNT(order_id) AS order_count
117 FROM orders
```

The result grid at the bottom shows the output of the query:

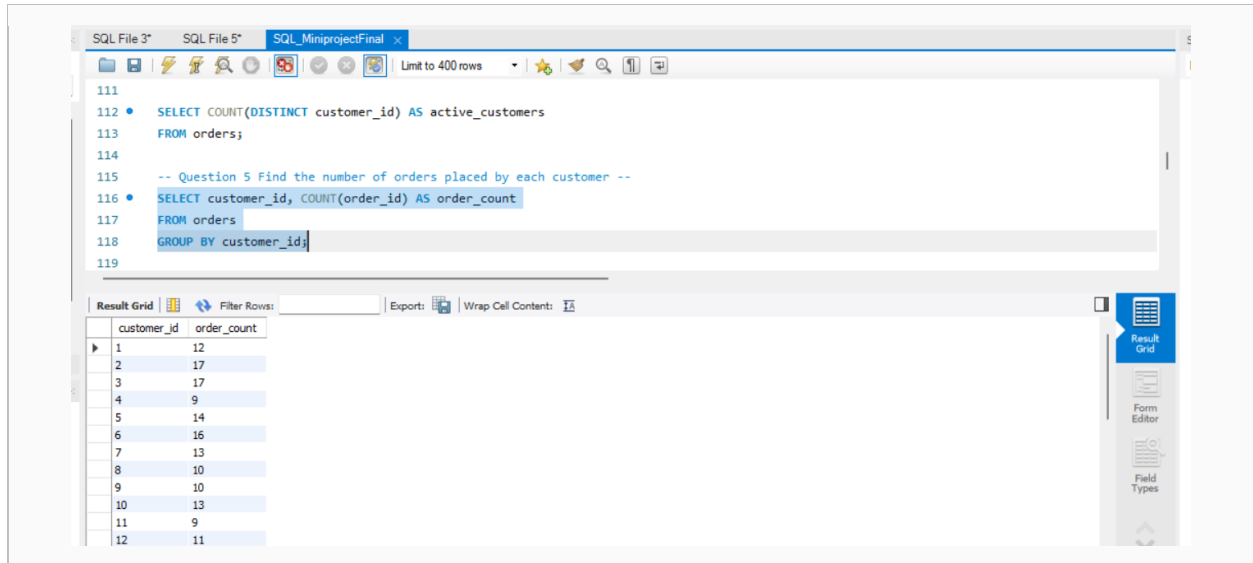
active_customers
30

Question 5

Find the number of orders placed by each customer

```
SELECT customer_id, COUNT(order_id) AS order_count
FROM orders
GROUP BY customer_id;
```

Output:



The screenshot shows a SQL query editor with the following code:

```
111
112 • SELECT COUNT(DISTINCT customer_id) AS active_customers
113 FROM orders;
114
115 -- Question 5 Find the number of orders placed by each customer --
116 • SELECT customer_id, COUNT(order_id) AS order_count
117 FROM orders
118 GROUP BY customer_id;
119
```

Below the editor is a 'Result Grid' showing the output of the query. The grid has two columns: 'customer_id' and 'order_count'. The data is as follows:

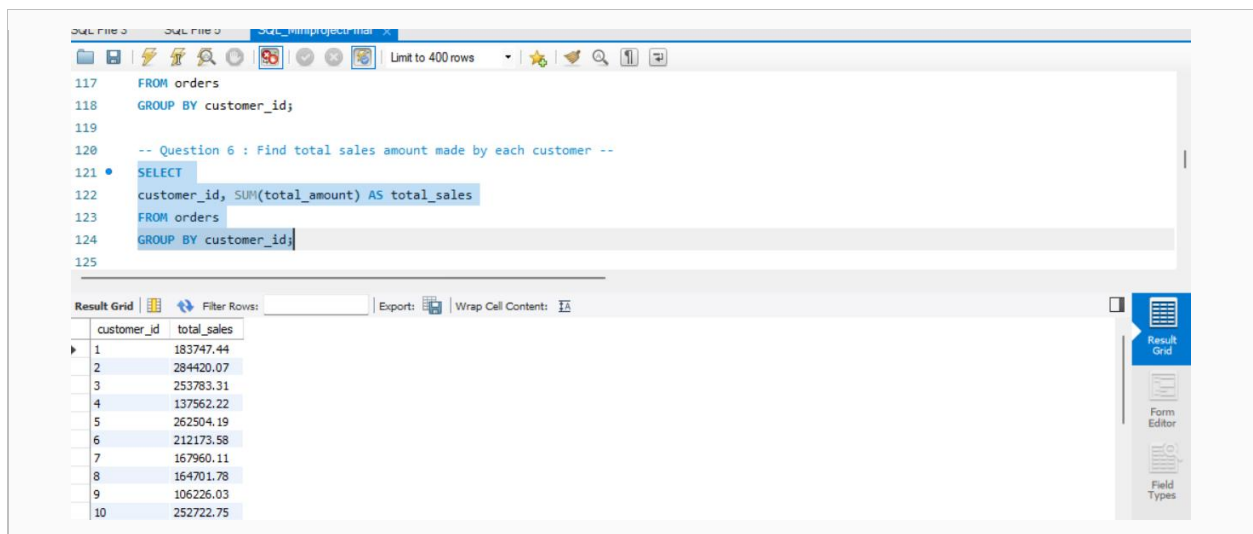
customer_id	order_count
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10
10	13
11	9
12	11

Question 6

Find total sales amount made by each customer

```
SELECT customer_id, SUM(total_amount) AS total_sales
FROM orders
GROUP BY customer_id;
```

Output:



The screenshot shows a SQL query editor with the following code:

```
117 FROM orders
118 GROUP BY customer_id;
119
120 -- Question 6 : Find total sales amount made by each customer --
121 • SELECT
122 customer_id, SUM(total_amount) AS total_sales
123 FROM orders
124 GROUP BY customer_id;
125
```

Below the editor is a 'Result Grid' showing the output of the query. The grid has two columns: 'customer_id' and 'total_sales'. The data is as follows:

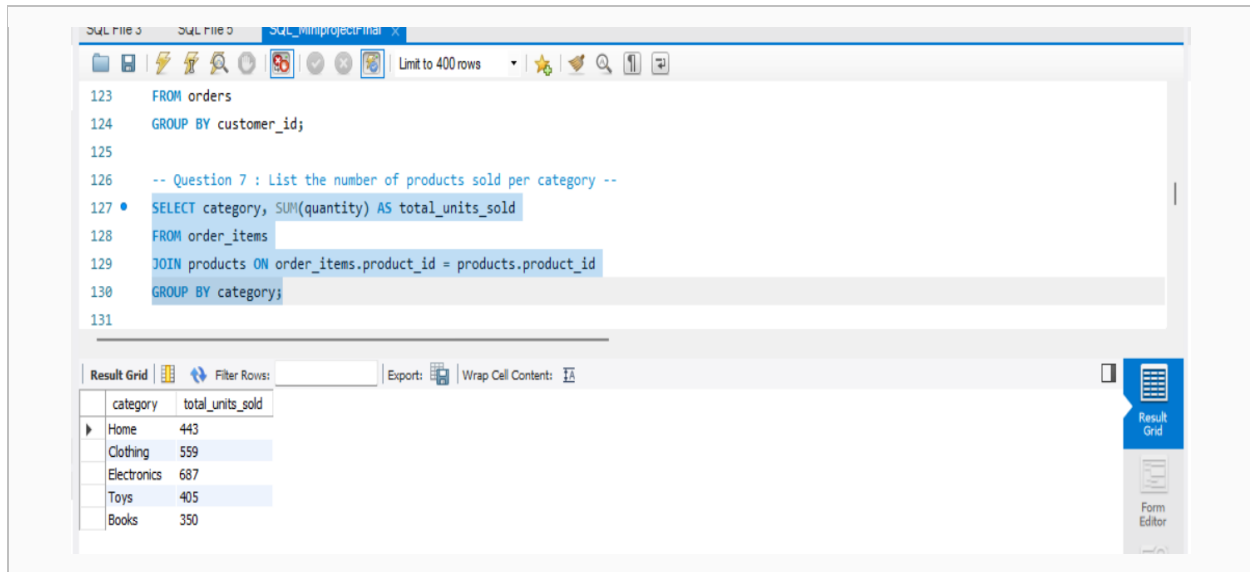
customer_id	total_sales
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75

Question 7

List the number of products sold per category

```
SELECT category, SUM(quantity) AS total_units_sold
FROM order_items
JOIN products ON order_items.product_id = products.product_id
GROUP BY category;
```

Output:



The screenshot shows a SQL IDE window with the following query:

```
123 FROM orders
124 GROUP BY customer_id;
125
126 -- Question 7 : List the number of products sold per category --
127 • SELECT category, SUM(quantity) AS total_units_sold
128 FROM order_items
129 JOIN products ON order_items.product_id = products.product_id
130 GROUP BY category;
131
```

The output is displayed in a table with the following data:

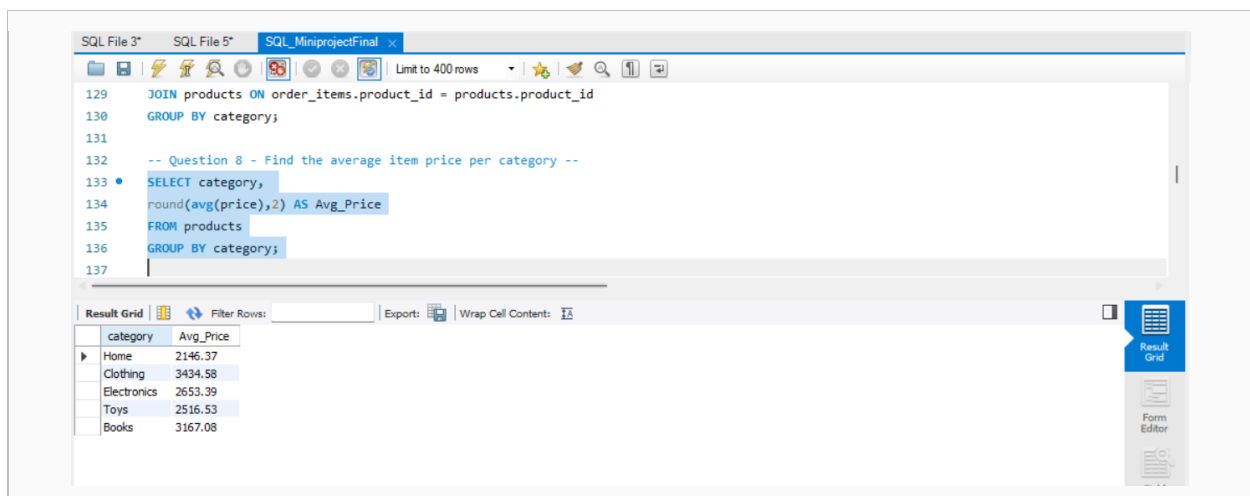
category	total_units_sold
Home	443
Clothing	559
Electronics	687
Toys	405
Books	350

Question 8

Find the average item price per category

```
SELECT category, ROUND(AVG(price),2) AS Avg_Price
FROM products
GROUP BY category;
```

Output:



The screenshot shows a SQL IDE window with the following query:

```
129 JOIN products ON order_items.product_id = products.product_id
130 GROUP BY category;
131
132 -- Question 8 - Find the average item price per category --
133 • SELECT category,
134 round(avg(price),2) AS Avg_Price
135 FROM products
136 GROUP BY category;
137
```

The output is displayed in a table with the following data:

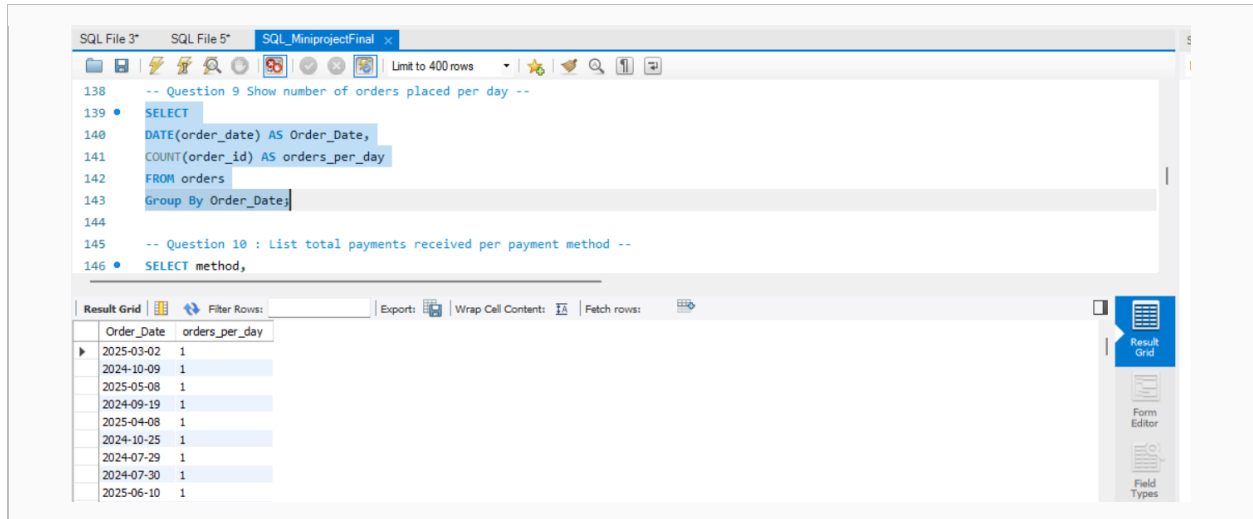
category	Avg_Price
Home	2146.37
Clothing	3434.58
Electronics	2653.39
Toys	2516.53
Books	3167.08

Question 9

Show number of orders placed per day

```
SELECT DATE(order_date) AS Order_Date, COUNT(order_id) AS orders_per_day
FROM orders
GROUP BY Order_Date;
```

Output:



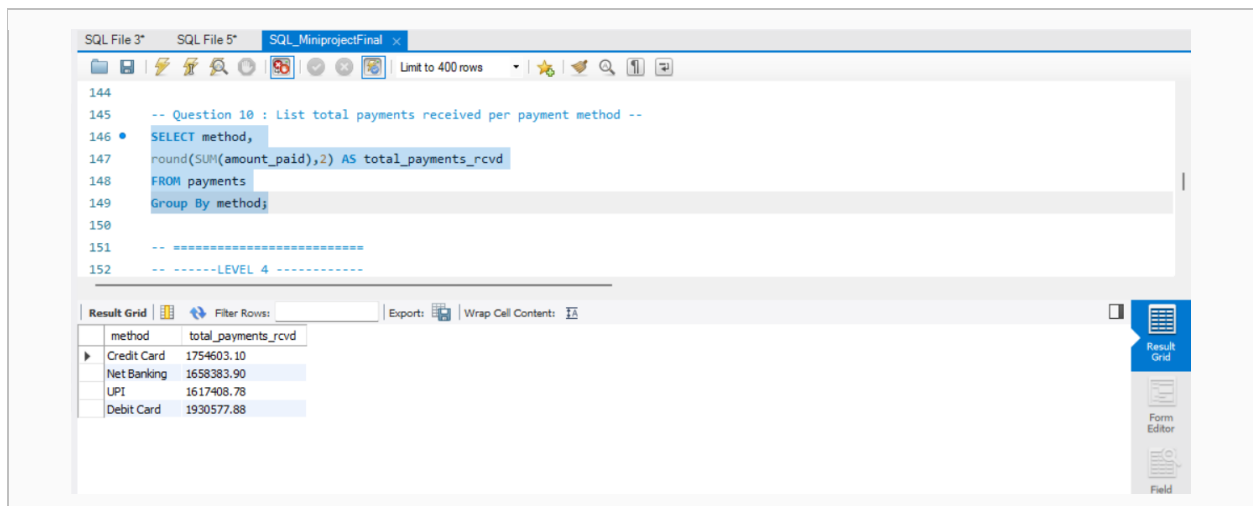
Order_Date	orders_per_day
2025-03-02	1
2024-10-09	1
2025-05-08	1
2024-09-19	1
2025-04-08	1
2024-10-25	1
2024-07-29	1
2024-07-30	1
2025-06-10	1

Question 10

List total payments received per payment method

```
SELECT method, ROUND(SUM(amount_paid),2) AS total_payments_rcvd
FROM payments
GROUP BY method;
```

Output:



method	total_payments_rcvd
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78
Debit Card	1930577.88

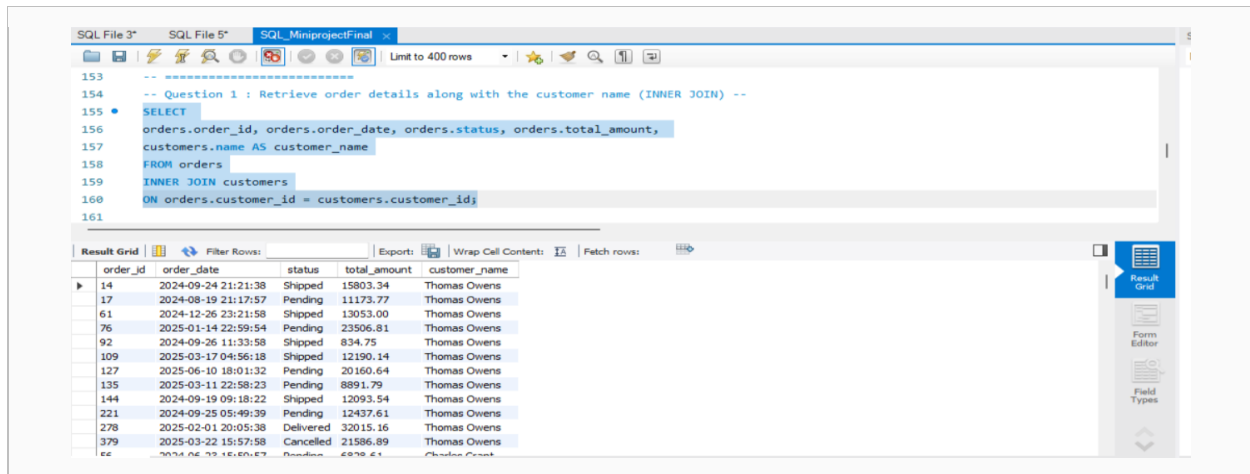
Level 4 — Multi-Table JOINS

Question 1

Retrieve order details along with the customer name (INNER JOIN)

```
SELECT orders.order_id, orders.order_date, orders.status, orders.total_amount,
customers.name AS customer_name
FROM orders
INNER JOIN customers ON orders.customer_id = customers.customer_id;
```

Output:



The screenshot shows a SQL IDE window with a query editor and a results grid. The query is an INNER JOIN between the 'orders' and 'customers' tables. The results grid displays 14 rows of data, including order_id, order_date, status, total_amount, and customer_name.

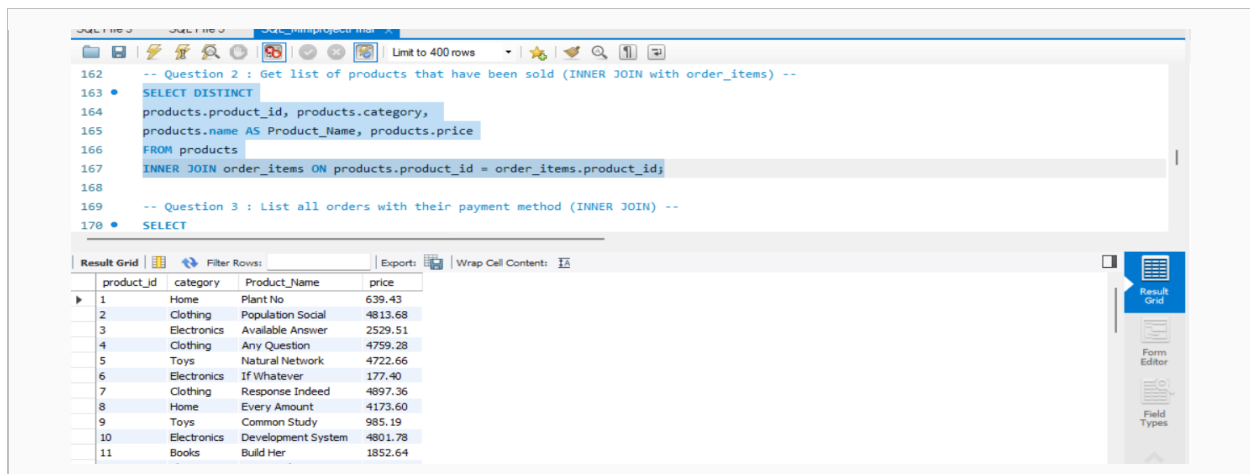
order_id	order_date	status	total_amount	customer_name
14	2024-09-24 21:21:38	Shipped	15803.34	Thomas Owens
17	2024-08-19 21:17:57	Pending	11173.77	Thomas Owens
61	2024-12-26 23:21:58	Shipped	13053.00	Thomas Owens
76	2025-01-14 22:59:54	Pending	23506.81	Thomas Owens
92	2024-09-26 11:33:58	Shipped	834.75	Thomas Owens
109	2025-03-17 04:56:18	Shipped	12190.14	Thomas Owens
127	2025-06-10 18:01:32	Pending	20160.64	Thomas Owens
135	2025-03-11 22:58:23	Pending	8891.79	Thomas Owens
144	2024-09-19 09:18:22	Shipped	12093.54	Thomas Owens
221	2024-09-25 05:49:39	Pending	12437.61	Thomas Owens
278	2025-02-01 20:05:38	Delivered	32015.16	Thomas Owens
379	2025-03-22 15:57:58	Cancelled	21586.89	Thomas Owens
44	2024-06-23 16:01:57	Pending	6890.61	Charles Grant

Question 2

Get list of products that have been sold (INNER JOIN with order_items)

```
SELECT DISTINCT products.product_id, products.category,
products.name AS Product_Name, products.price
FROM products
INNER JOIN order_items ON products.product_id = order_items.product_id;
```

Output:



The screenshot shows a SQL IDE window with a query editor and a results grid. The query is an INNER JOIN between the 'products' and 'order_items' tables, using DISTINCT to show unique products. The results grid displays 11 rows of data, including product_id, category, Product_Name, and price.

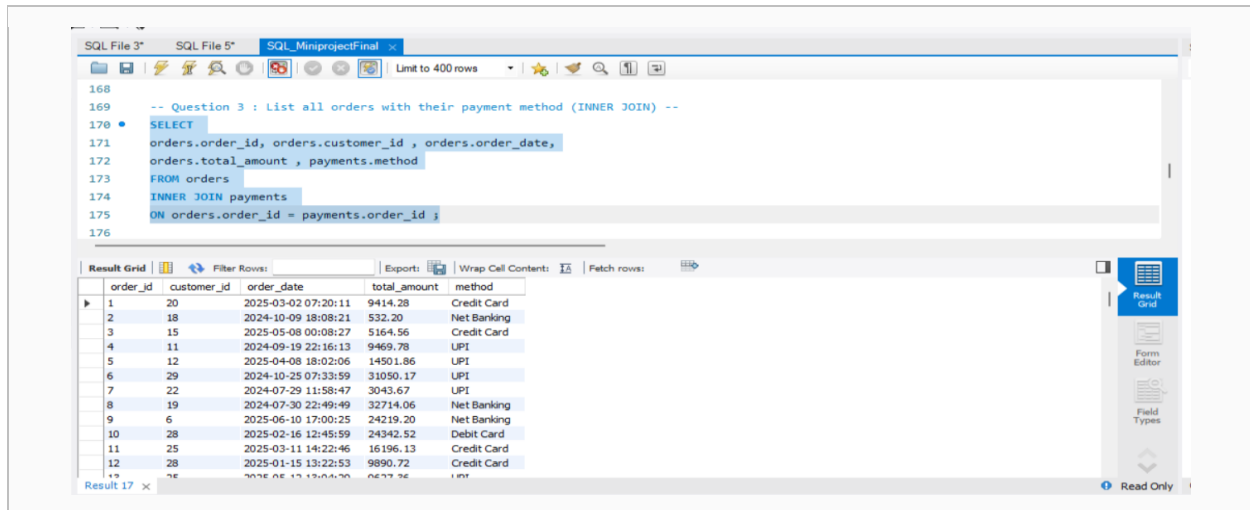
product_id	category	Product_Name	price
1	Home	Plant No	639.43
2	Clothing	Population Social	4813.68
3	Electronics	Available Answer	2529.51
4	Clothing	Any Question	4759.28
5	Toys	Natural Network	4722.66
6	Electronics	If Whatever	177.40
7	Clothing	Response Indeed	4897.36
8	Home	Every Amount	4173.60
9	Toys	Common Study	985.19
10	Electronics	Development System	4801.78
11	Books	Build Her	1852.64

Question 3

List all orders with their payment method (INNER JOIN)

```
SELECT orders.order_id, orders.customer_id, orders.order_date,  
orders.total_amount, payments.method  
FROM orders  
INNER JOIN payments ON orders.order_id = payments.order_id;
```

Output:



The screenshot shows the SQL Studio interface. The SQL editor contains the following query:

```
-- Question 3 : List all orders with their payment method (INNER JOIN) --  
SELECT  
orders.order_id, orders.customer_id, orders.order_date,  
orders.total_amount, payments.method  
FROM orders  
INNER JOIN payments  
ON orders.order_id = payments.order_id ;
```

The result grid displays the following data:

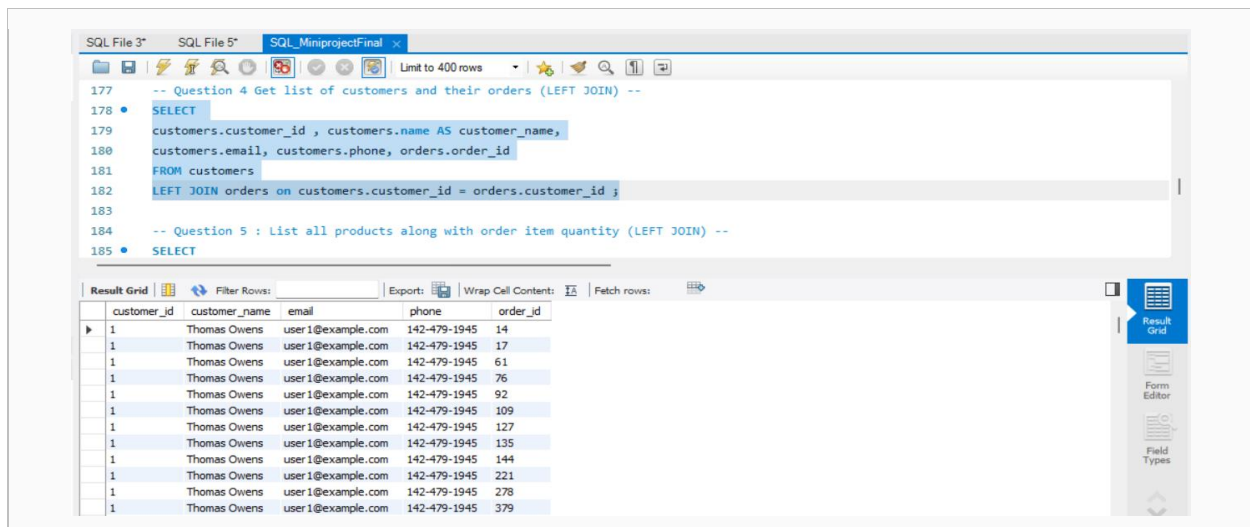
order_id	customer_id	order_date	total_amount	method
1	20	2025-03-02 07:20:11	9414.28	Credit Card
2	18	2024-10-09 18:08:21	532.20	Net Banking
3	15	2025-05-08 00:08:27	5164.56	Credit Card
4	11	2024-09-19 22:16:13	9469.78	UPI
5	12	2025-04-08 18:02:06	1450.86	UPI
6	29	2024-10-25 07:33:59	31050.17	UPI
7	22	2024-07-29 11:58:47	3043.67	UPI
8	19	2024-07-30 22:49:49	32714.06	Net Banking
9	6	2025-06-10 17:00:25	24219.20	Net Banking
10	28	2025-02-16 12:45:59	24342.52	Debit Card
11	25	2025-03-11 14:22:46	16196.13	Credit Card
12	28	2025-01-15 13:22:53	9890.72	Credit Card

Question 4

Get list of customers and their orders (LEFT JOIN)

```
SELECT customers.customer_id, customers.name AS customer_name,  
customers.email, customers.phone, orders.order_id  
FROM customers  
LEFT JOIN orders ON customers.customer_id = orders.customer_id;
```

Output:



The screenshot shows the SQL Studio interface. The SQL editor contains the following query:

```
-- Question 4 Get list of customers and their orders (LEFT JOIN) --  
SELECT  
customers.customer_id, customers.name AS customer_name,  
customers.email, customers.phone, orders.order_id  
FROM customers  
LEFT JOIN orders on customers.customer_id = orders.customer_id ;
```

The result grid displays the following data:

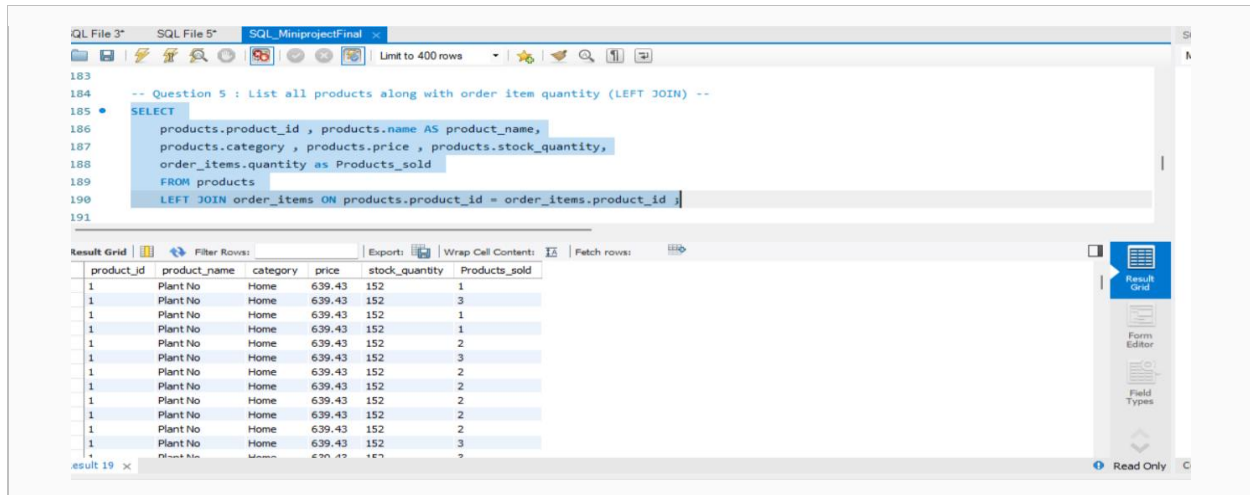
customer_id	customer_name	email	phone	order_id
1	Thomas Owens	user1@example.com	142-479-1945	14
1	Thomas Owens	user1@example.com	142-479-1945	17
1	Thomas Owens	user1@example.com	142-479-1945	61
1	Thomas Owens	user1@example.com	142-479-1945	76
1	Thomas Owens	user1@example.com	142-479-1945	92
1	Thomas Owens	user1@example.com	142-479-1945	109
1	Thomas Owens	user1@example.com	142-479-1945	127
1	Thomas Owens	user1@example.com	142-479-1945	135
1	Thomas Owens	user1@example.com	142-479-1945	144
1	Thomas Owens	user1@example.com	142-479-1945	221
1	Thomas Owens	user1@example.com	142-479-1945	278
1	Thomas Owens	user1@example.com	142-479-1945	379

Question 5

List all products along with order item quantity (LEFT JOIN)

```
SELECT products.product_id, products.name AS product_name,  
products.category, products.price, products.stock_quantity,  
order_items.quantity AS Products_sold  
FROM products  
LEFT JOIN order_items ON products.product_id = order_items.product_id;
```

Output:



```
-- Question 5 : List all products along with order item quantity (LEFT JOIN) --  
SELECT  
products.product_id , products.name AS product_name,  
products.category , products.price , products.stock_quantity,  
order_items.quantity as Products_sold  
FROM products  
LEFT JOIN order_items ON products.product_id = order_items.product_id ;
```

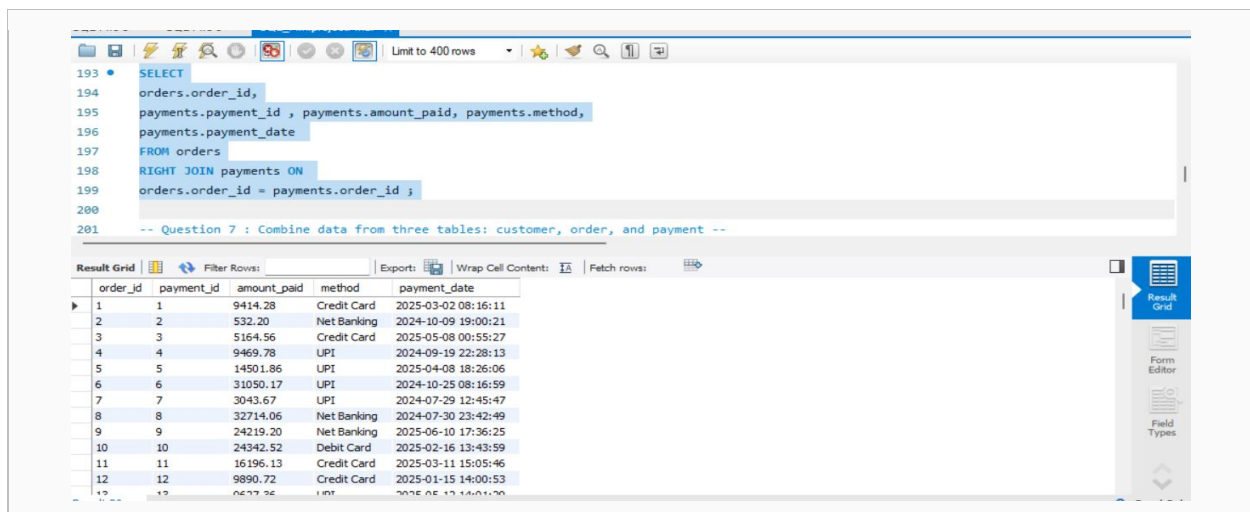
product_id	product_name	category	price	stock_quantity	Products_sold
1	Plant No	Home	639.43	152	1
1	Plant No	Home	639.43	152	3
1	Plant No	Home	639.43	152	1
1	Plant No	Home	639.43	152	1
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	3
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	3
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	3
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	2
1	Plant No	Home	639.43	152	2

Question 6

List all payments including those with no matching orders (RIGHT JOIN)

```
SELECT orders.order_id,  
payments.payment_id, payments.amount_paid, payments.method, payments.payment_date  
FROM orders  
RIGHT JOIN payments ON  
orders.order_id = payments.order_id;
```

Output:



```
SELECT  
orders.order_id,  
payments.payment_id , payments.amount_paid, payments.method,  
payments.payment_date  
FROM orders  
RIGHT JOIN payments ON  
orders.order_id = payments.order_id ;
```

```
-- Question 7 : Combine data from three tables: customer, order, and payment --
```

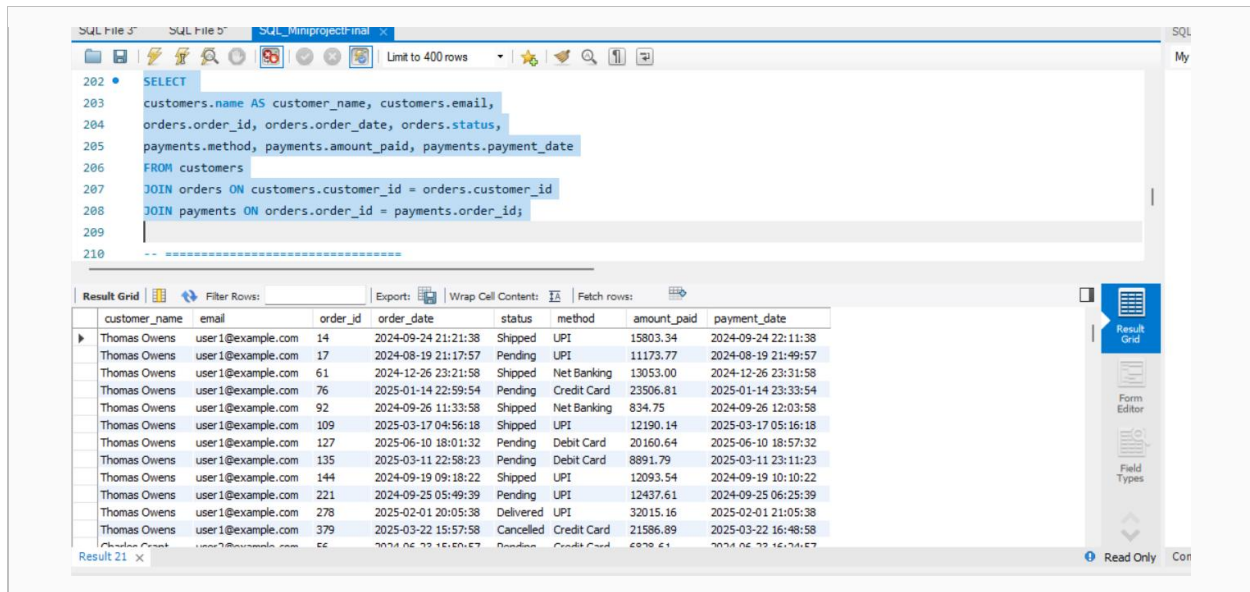
order_id	payment_id	amount_paid	method	payment_date
1	1	9414.28	Credit Card	2025-03-02 08:16:11
2	2	532.20	Net Banking	2024-10-09 19:00:21
3	3	5164.56	Credit Card	2025-05-08 00:55:27
4	4	9469.78	UPI	2024-09-19 22:28:13
5	5	14501.86	UPI	2025-04-08 18:26:06
6	6	31050.17	UPI	2024-10-25 08:16:59
7	7	3043.67	UPI	2024-07-29 12:45:47
8	8	32714.06	Net Banking	2024-07-30 23:42:49
9	9	24219.20	Net Banking	2025-06-10 17:36:25
10	10	24342.52	Debit Card	2025-02-16 13:43:59
11	11	16196.13	Credit Card	2025-03-11 15:05:46
12	12	9890.72	Credit Card	2025-01-15 14:00:53
13	13	9677.76	UPI	2025-05-13 14:01:30

Question 7

Combine data from three tables: customer, order, and payment

```
SELECT customers.name AS customer_name, customers.email,  
orders.order_id, orders.order_date, orders.status,  
payments.method, payments.amount_paid, payments.payment_date  
FROM customers  
JOIN orders ON customers.customer_id = orders.customer_id  
JOIN payments ON orders.order_id = payments.order_id;
```

Output:



The screenshot shows a SQL IDE interface with a query editor at the top and a results grid at the bottom. The query editor contains the following SQL code:

```
202 • SELECT  
203 customers.name AS customer_name, customers.email,  
204 orders.order_id, orders.order_date, orders.status,  
205 payments.method, payments.amount_paid, payments.payment_date  
206 FROM customers  
207 JOIN orders ON customers.customer_id = orders.customer_id  
208 JOIN payments ON orders.order_id = payments.order_id;  
209  
210
```

The results grid displays the output of the query, showing columns: customer_name, email, order_id, order_date, status, method, amount_paid, and payment_date. The data is as follows:

customer_name	email	order_id	order_date	status	method	amount_paid	payment_date
Thomas Owens	user1@example.com	14	2024-09-24 21:21:38	Shipped	UPI	15803.34	2024-09-24 22:11:38
Thomas Owens	user1@example.com	17	2024-08-19 21:17:57	Pending	UPI	11173.77	2024-08-19 21:49:57
Thomas Owens	user1@example.com	61	2024-12-26 23:21:58	Shipped	Net Banking	13053.00	2024-12-26 23:31:58
Thomas Owens	user1@example.com	76	2025-01-14 22:59:54	Pending	Credit Card	23506.81	2025-01-14 23:33:54
Thomas Owens	user1@example.com	92	2024-09-26 11:33:58	Shipped	Net Banking	834.75	2024-09-26 12:03:58
Thomas Owens	user1@example.com	109	2025-03-17 04:56:18	Shipped	UPI	12190.14	2025-03-17 05:16:18
Thomas Owens	user1@example.com	127	2025-06-10 18:01:32	Pending	Debit Card	20160.64	2025-06-10 18:57:32
Thomas Owens	user1@example.com	135	2025-03-11 22:58:23	Pending	Debit Card	8891.79	2025-03-11 23:11:23
Thomas Owens	user1@example.com	144	2024-09-19 09:18:22	Shipped	UPI	12093.54	2024-09-19 10:10:22
Thomas Owens	user1@example.com	221	2024-09-25 05:49:39	Pending	UPI	12437.61	2024-09-25 06:25:39
Thomas Owens	user1@example.com	278	2025-02-01 20:05:38	Delivered	UPI	32015.16	2025-02-01 21:05:38
Thomas Owens	user1@example.com	379	2025-03-22 15:57:58	Cancelled	Credit Card	21586.89	2025-03-22 16:48:58
Thomas Owens	user1@example.com	56	2024-06-22 15:50:57	Pending	Credit Card	6878.61	2024-06-22 16:24:57

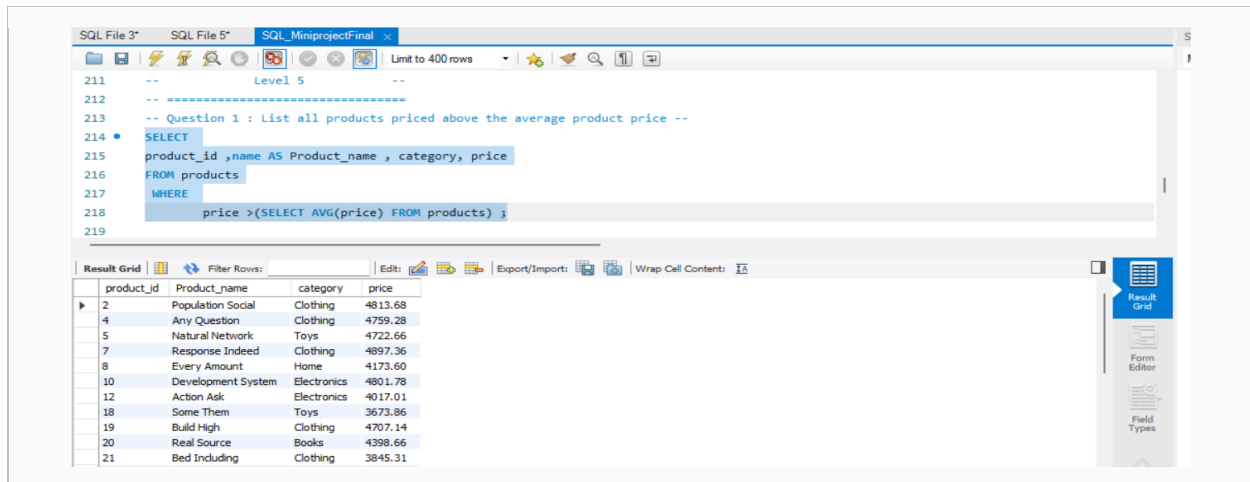
Level 5 — Subqueries

Question 1

List all products priced above the average product price

```
SELECT product_id, name AS Product_name, category, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Output:



The screenshot shows a SQL editor with the following query:

```
-- Level 5 --
-- Question 1 : List all products priced above the average product price --
SELECT
product_id ,name AS Product_name , category, price
FROM products
WHERE
price >(SELECT AVG(price) FROM products) ;
```

The results grid displays the following data:

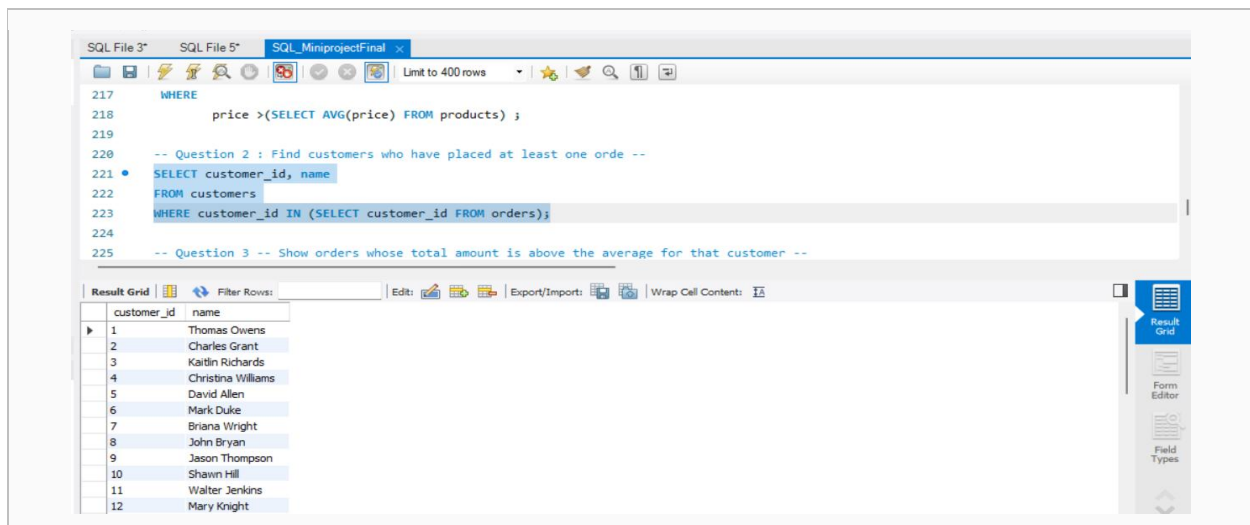
product_id	Product_name	category	price
2	Population Social	Clothing	4813.68
4	Any Question	Clothing	4759.28
5	Natural Network	Toys	4722.66
7	Response Indeed	Clothing	4897.36
8	Every Amount	Home	4173.60
10	Development System	Electronics	4801.78
12	Action Ask	Electronics	4017.01
18	Some Them	Toys	3673.86
19	Build High	Clothing	4707.14
20	Real Source	Books	4398.66
21	Bed Including	Clothing	3845.31

Question 2

Find customers who have placed at least one order

```
SELECT customer_id, name
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);
```

Output:



The screenshot shows a SQL editor with the following query:

```
WHERE
price >(SELECT AVG(price) FROM products) ;
-- Question 2 : Find customers who have placed at least one orde --
SELECT customer_id, name
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);
-- Question 3 -- Show orders whose total amount is above the average for that customer --
```

The results grid displays the following data:

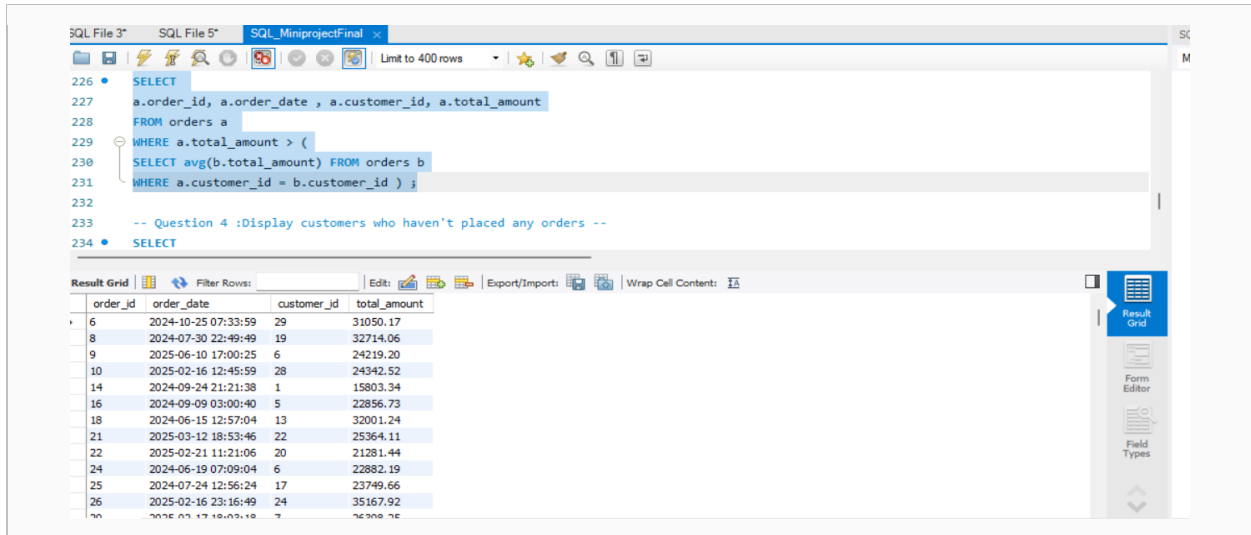
customer_id	name
1	Thomas Owens
2	Charles Grant
3	Kaitlin Richards
4	Christina Williams
5	David Allen
6	Mark Duke
7	Briana Wright
8	John Bryan
9	Jason Thompson
10	Shawn Hill
11	Walter Jenkins
12	Mary Knight

Question 3

Show orders whose total amount is above the average for that customer

```
SELECT a.order_id, a.order_date, a.customer_id, a.total_amount
FROM orders a
WHERE a.total_amount > (
    SELECT AVG(b.total_amount) FROM orders b
    WHERE a.customer_id = b.customer_id
);
```

Output:



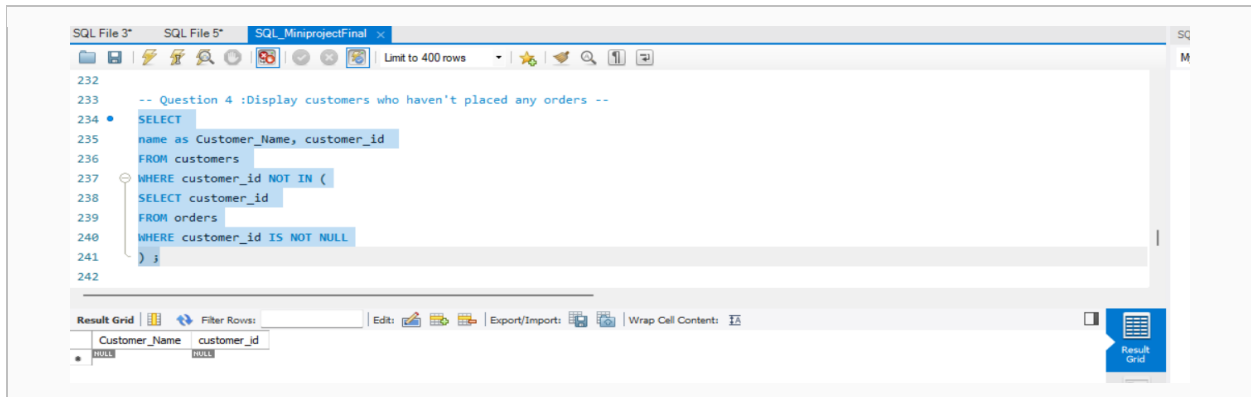
order_id	order_date	customer_id	total_amount
6	2024-10-25 07:33:59	29	31050.17
8	2024-07-30 22:49:49	19	32714.06
9	2025-06-10 17:00:25	6	24219.20
10	2025-02-16 12:45:59	28	24342.52
14	2024-09-24 21:21:38	1	15803.34
16	2024-09-09 03:00:40	5	22856.73
18	2024-06-15 12:57:04	13	32001.24
21	2025-03-12 18:53:46	22	25364.11
22	2025-02-21 11:21:06	20	21281.44
24	2024-06-19 07:09:04	6	22892.19
25	2024-07-24 12:56:24	17	23749.66
26	2025-02-16 23:16:49	24	35167.92

Question 4

Display customers who haven't placed any orders

```
SELECT name AS Customer_Name, customer_id
FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders WHERE customer_id IS NOT NULL
);
```

Output:



Customer_Name	customer_id
NULL	NULL
NULL	NULL

Question 5

Show products that were never ordered

```
SELECT product_id, name AS Product_name, category
FROM products
WHERE product_id NOT IN (SELECT product_id FROM order_items);
```

Output:

The screenshot shows the SQL Studio interface with the following SQL query:

```
-- Question 5 : Show products that were never ordered --
SELECT
product_id, name AS Product_name, category
FROM products
WHERE product_id NOT IN (SELECT product_id FROM order_items);
```

The result grid below the query shows the following data:

product_id	Product_name	category
1	...	...
2	...	...
3	...	...
4	...	...
5	...	...
6	...	...
7	...	...
8	...	...
9	...	...

Question 6

Show highest value order per customer

```
SELECT customer_id, MAX(total_amount) AS highest_order_value
FROM orders
GROUP BY customer_id;
```

Output:

The screenshot shows the SQL Studio interface with the following SQL query:

```
-- Question 6 : Show highest value order per customer --
SELECT customer_id, MAX(total_amount) AS highest_order_value
FROM orders
GROUP BY customer_id;
```

The result grid below the query shows the following data:

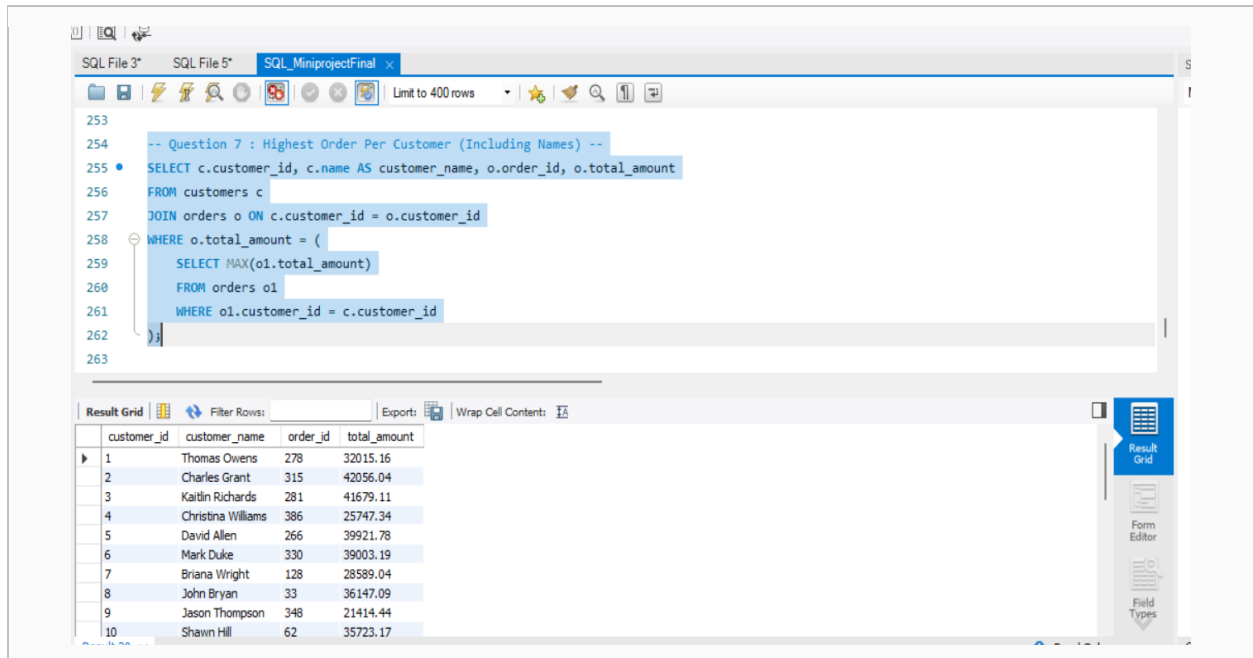
customer_id	highest_order_value
1	32015.16
2	42056.04
3	41679.11
4	25747.34
5	39921.78
6	39003.19
7	28589.04
8	36147.09
9	21414.44

Question 7

Highest Order Per Customer (Including Names)

```
SELECT c.customer_id, c.name AS customer_name, o.order_id, o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.total_amount = (
    SELECT MAX(o1.total_amount)
    FROM orders o1
    WHERE o1.customer_id = c.customer_id
);
```

Output:



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
-- Question 7 : Highest Order Per Customer (Including Names) --
SELECT c.customer_id, c.name AS customer_name, o.order_id, o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.total_amount = (
    SELECT MAX(o1.total_amount)
    FROM orders o1
    WHERE o1.customer_id = c.customer_id
);
```

The results grid displays the following data:

	customer_id	customer_name	order_id	total_amount
1	1	Thomas Owens	278	32015.16
2	2	Charles Grant	315	42056.04
3	3	Kaitlin Richards	281	41679.11
4	4	Christina Williams	386	25747.34
5	5	David Allen	266	39921.78
6	6	Mark Duke	330	39003.19
7	7	Briana Wright	128	28589.04
8	8	John Bryan	33	36147.09
9	9	Jason Thompson	348	21414.44
10	10	Shawn Hill	62	35723.17

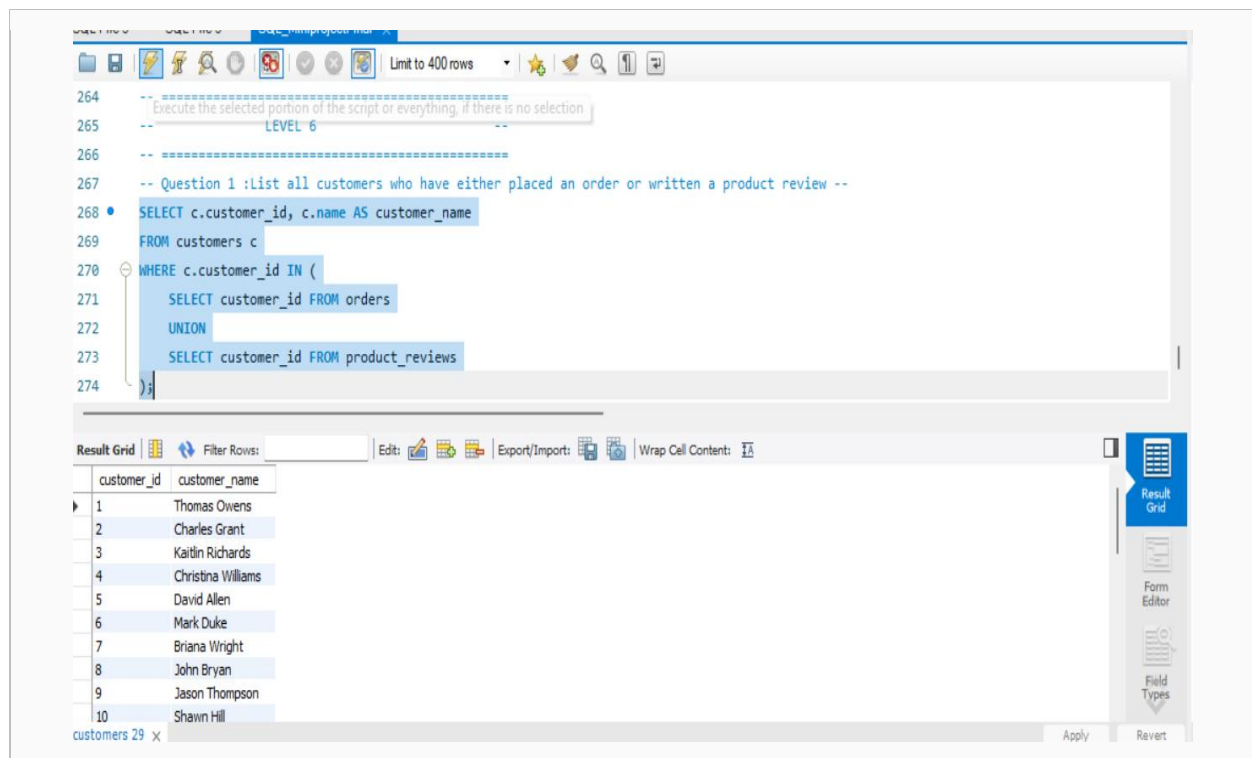
Level 6 — Set Operations

Question 1

List all customers who have either placed an order or written a product review

```
SELECT c.customer_id, c.name AS customer_name
FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id FROM orders
    UNION
    SELECT customer_id FROM product_reviews
);
```

Output:



The screenshot shows a SQL IDE interface. The top pane displays a SQL query with line numbers 264 to 274. The query is a SELECT statement with a subquery in the WHERE clause, using UNION to combine results from the orders and product_reviews tables. The bottom pane shows the 'Result Grid' with two columns: 'customer_id' and 'customer_name'. It lists 10 rows of data. The interface includes a toolbar at the top with icons for file operations, a 'Limit to 400 rows' dropdown, and a 'Filter Rows' field. The bottom right has 'Apply' and 'Revert' buttons.

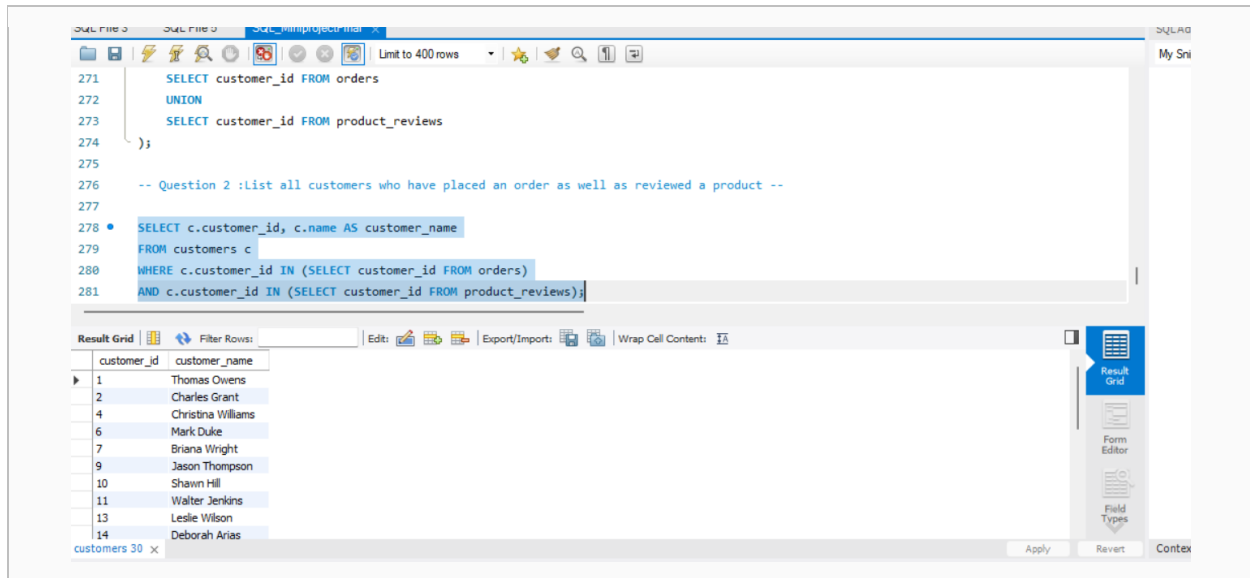
customer_id	customer_name
1	Thomas Owens
2	Charles Grant
3	Kaitlin Richards
4	Christina Williams
5	David Allen
6	Mark Duke
7	Briana Wright
8	John Bryan
9	Jason Thompson
10	Shawn Hill

Question 2

List all customers who have placed an order as well as reviewed a product

```
SELECT c.customer_id, c.name AS customer_name
FROM customers c
WHERE c.customer_id IN (SELECT customer_id FROM orders)
AND c.customer_id IN (SELECT customer_id FROM product_reviews);
```

Output:



The screenshot shows a SQL IDE interface. The top pane displays the following SQL query:

```
271 SELECT customer_id FROM orders
272 UNION
273 SELECT customer_id FROM product_reviews
274 );
275
276 -- Question 2 :List all customers who have placed an order as well as reviewed a product --
277
278 • SELECT c.customer_id, c.name AS customer_name
279 FROM customers c
280 WHERE c.customer_id IN (SELECT customer_id FROM orders)
281 AND c.customer_id IN (SELECT customer_id FROM product_reviews);
```

The bottom pane shows the 'Result Grid' with the following data:

customer_id	customer_name
1	Thomas Owens
2	Charles Grant
4	Christina Williams
6	Mark Duke
7	Briana Wright
9	Jason Thompson
10	Shawn Hill
11	Walter Jenkins
13	Leslie Wilson
14	Deborah Arias

The interface includes a toolbar at the top with icons for file operations, a 'Limit to 400 rows' dropdown, and a 'My Sni' sidebar on the right. The bottom right corner has 'Apply', 'Revert', and 'Context' buttons.